glmCH2_clean

Load and clean data

```
#load packages
library(tidyverse) #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMa) #diagnostic tools for mixed regression models
```

```
## This is DHARMa 0.4.7. For overview type '?DHARMa'. For recent changes, type news(package = 'DHARMa')
```

```
library(glmmTMB) #fits generalized linear mixed models, including zero-inflated and overdispersed models
library(car) #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(emmeans)   # for estimated marginal means (predictions)
```

```
## Welcome to emmeans.
## Caution: You lose important information if you filter this package's results.
## See '? untidy'
```

```r
library(lubridate) #date <-> julian date

#read in data
data <- read.csv("06_cleandata/combined_plot_data.csv")

#change character columns to factors
surveys <- data %>%
   mutate(
     site_id = as.factor(site_id),
     round = as.factor(round),
     plot_id = as.factor(plot_id),
     bee_date = as.Date(bee_date),
     bee_season_type = as.factor(bee_season_type),
     area = as.factor(area),
     cover_type = as.factor(cover_type))

#add plot type column and make it a factor
surveys <- surveys %>%
  mutate(plot_type = if_else(plot_id == "d", "ditch", "field"))
surveys <- surveys %>%
   mutate(
     plot_type = as.factor(plot_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))
surveys$num_inactive_burrows_scaled <- as.numeric(scale(surveys$num_inactive_burrows))
surveys$num_active_dm_scaled <- as.numeric(scale(surveys$num_active_dm))
surveys$num_dm_size_scaled <- as.numeric(scale(surveys$num_dm_size))
surveys$num_inactive_dm_scaled <- as.numeric(scale(surveys$num_inactive_dm))
```

Choosing which vegetation variables to include as predictors

1. Use PCA to determine which vegetation variables to include in my models. I don't want to include all of the predictors, since some will be correlated with each other and therefore dilute their effects of the response variables. I am using PCA so I can pick one variable that drives each of the main 3 components, and use these as the predictors in my model.

```
pca_res <- prcomp(surveys[, c("bare_perc_scaled",
                              "grass_forb_perc_scaled",
                              "grass_h_scaled",
                              "forb_h_scaled",
                              "bombus_floral_perc_scaled",
                              "flood_perc_scaled",
                              "dry_root_perc_scaled",
                              "bombus_floral_richness_scaled",
                              "bombus_floral_abun_scaled")], scale. = TRUE)
summary(pca_res)
```

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     1.8037 1.2128 1.0888 0.94326 0.90070 0.82536 0.54028
## Proportion of Variance 0.3615 0.1634 0.1317 0.09886 0.09014 0.07569 0.03243
## Cumulative Proportion  0.3615 0.5249 0.6566 0.75551 0.84565 0.92135 0.95378
##                           PC8     PC9
## Standard deviation     0.5187 0.38329
## Proportion of Variance 0.0299 0.01632
## Cumulative Proportion  0.9837 1.00000
```

```
#see which variable drives each of the principle components
print(pca_res$rotation)
```

```
##                                    PC1         PC2        PC3          PC4
## bare_perc_scaled               0.24100694  0.51132677  0.1480093 -0.506337915
## grass_forb_perc_scaled        -0.42303293 -0.22390273 -0.3007926 -0.006568266
## grass_h_scaled                -0.49812518 -0.11060633 -0.0470433 -0.149976271
## forb_h_scaled                 -0.48569527 -0.08315431  0.1134057 -0.240186492
## bombus_floral_perc_scaled     -0.18883655  0.40010344 -0.1653710  0.651841412
## flood_perc_scaled              0.12306241 -0.07874923 -0.6378149  0.019972489
## dry_root_perc_scaled           0.02157359 -0.40268963  0.6227065  0.317142224
## bombus_floral_richness_scaled -0.43081001  0.26087490  0.1437940 -0.233788645
## bombus_floral_abun_scaled     -0.20509491  0.52226932  0.1730539  0.287860892
##                                    PC5          PC6         PC7         PC8
## bare_perc_scaled               0.06538925  0.370121563  0.49842251  0.10038801
## grass_forb_perc_scaled        -0.27778537 -0.153122311  0.54910763  0.48948701
## grass_h_scaled                 0.02851587  0.203518533  0.20705746 -0.38586034
## forb_h_scaled                  0.15759445  0.172355381 -0.08782852 -0.41865475
## bombus_floral_perc_scaled     -0.13014055  0.571974934 -0.03009873  0.03525875
## flood_perc_scaled              0.74841113  0.058657287  0.08301863  0.01827233
## dry_root_perc_scaled           0.42325486  0.207845797  0.28968027  0.20646515
## bombus_floral_richness_scaled  0.27097277  0.003296897 -0.48095986  0.58651301
## bombus_floral_abun_scaled      0.25156245 -0.628196361  0.27646719 -0.19483913
##                                    PC9
```

```
## bare_perc_scaled              -0.049064111
## grass_forb_perc_scaled        -0.196630307
## grass_h_scaled                 0.693496680
## forb_h_scaled                 -0.670154480
## bombus_floral_perc_scaled     -0.075975027
## flood_perc_scaled             -0.025721397
## dry_root_perc_scaled           0.012084553
## bombus_floral_richness_scaled  0.149185034
## bombus_floral_abun_scaled     -0.006968346
```

- PCA Results:
  - PC1: grass_h_scaled (-0.50),forb_h_scaled (-0.48),grass_forb_perc_scaled (-0.42),bombus_floral_richness_scal (-0.43),bombus_floral_abun_scaled (-0.22)
  - PC2: bare_perc_scaled (0.51),bombus_floral_abun_scaled (0.52),bombus_floral_perc_scaled (0.40), dry_root_perc_scaled (-0.40)
- Based on these PCA results and my research question, I am selecting to include the following predictors in my model:
  - grass_forb_perc_scaled (loads strongly on PCA1)
  - bombus_floral_abun_scaled (loads strongly on PCA1 &2)

3. Check pairwise correlations between this reduced set of variables

- correlation among all the pairs are all near zero –> predictors are all sufficiently independent! (R«0.7)

```
cor(surveys[, c("grass_forb_perc_scaled", "bombus_floral_abun_scaled")])
```

```
##                           grass_forb_perc_scaled bombus_floral_abun_scaled
## grass_forb_perc_scaled                1.00000000                0.07455522
## bombus_floral_abun_scaled             0.07455522                1.00000000
```

Comparing field and ditch plots at grassy and bare farms.

Vegetative cover

Steps

Try different random effects structures with gamma model

```
#adjust % grass and forb so that all values are positive
surveys$grass_forb_perc_adj <- surveys$grass_forb_perc + 0.001

#model with gamma distribution
ditchVSfield_veg_full <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                              (1|area/site_id/plot_id),
                            family = Gamma(link = "log"),
                            data = surveys)
```

```
ditchVSfield_veg_noArea <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                             (1|site_id/plot_id),
                           family = Gamma(link = "log"),
                           data = surveys)

ditchVSfield_veg_plotOnly <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                             (1|plot_id),
                           family = Gamma(link = "log"),
                           data = surveys)

ditchVSfield_veg_siteOnly <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                             (1|site_id),
                           family = Gamma(link = "log"),
                           data = surveys)

ditchVSfield_veg_siteArea <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                             (1|area/site_id),
                           family = Gamma(link = "log"),
                           data = surveys)

ditchVSfield_veg_areaPlot <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled +
                             (1|area/plot_id),
                           family = Gamma(link = "log"),
                           data = surveys)

ditchVSfield_veg_noRE <- glmmTMB(grass_forb_perc_adj ~ cover_type*plot_type + julian.date_scaled,
                           family = Gamma(link = "log"),
                           data = surveys)

#compare models
anova(ditchVSfield_veg_full, ditchVSfield_veg_noArea, ditchVSfield_veg_plotOnly, ditchVSfield_veg_siteOn
      ditchVSfield_veg_areaPlot, ditchVSfield_veg_noRE)
```

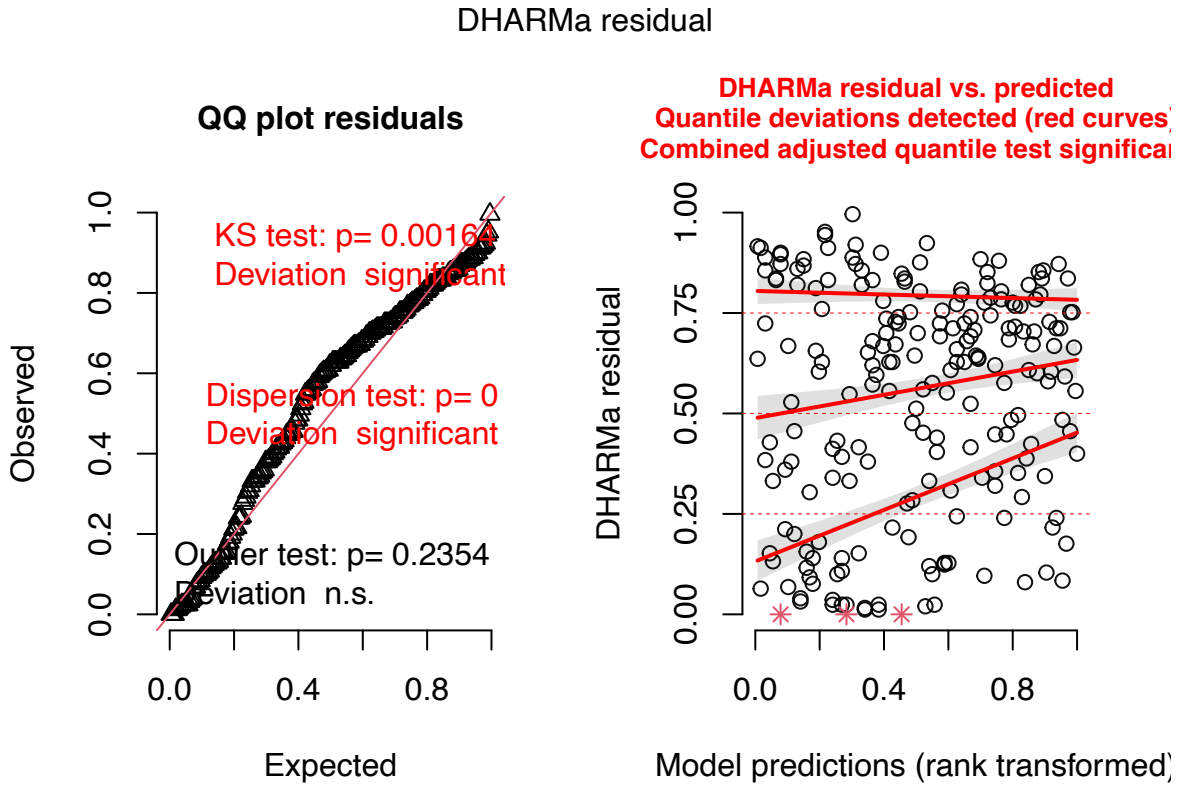
```
## Data: surveys
## Models:
## ditchVSfield_veg_noRE: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled, zi=~0, disp
## ditchVSfield_veg_plotOnly: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~
## ditchVSfield_veg_plotOnly:     (1 | plot_id), zi=~0, disp=~1
## ditchVSfield_veg_siteOnly: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~
## ditchVSfield_veg_siteOnly:     (1 | site_id), zi=~0, disp=~1
## ditchVSfield_veg_noArea: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~0
## ditchVSfield_veg_noArea:     (1 | site_id/plot_id), zi=~0, disp=~1
## ditchVSfield_veg_siteArea: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~
## ditchVSfield_veg_siteArea:     (1 | area/site_id), zi=~0, disp=~1
## ditchVSfield_veg_areaPlot: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~
## ditchVSfield_veg_areaPlot:     (1 | area/plot_id), zi=~0, disp=~1
## ditchVSfield_veg_full: grass_forb_perc_adj ~ cover_type * plot_type + julian.date_scaled + , zi=~0, d
## ditchVSfield_veg_full:     (1 | area/site_id/plot_id), zi=~0, disp=~1
##                          Df    AIC    BIC  logLik deviance   Chisq Chi Df
## ditchVSfield_veg_noRE      6 1847.1 1867.2 -917.55   1835.1
## ditchVSfield_veg_plotOnly  7 1849.1 1872.5 -917.55   1835.1  0.0000      1
## ditchVSfield_veg_siteOnly  7 1832.8 1856.2 -909.38   1818.8 16.3323      0
## ditchVSfield_veg_noArea    8 1834.8 1861.5 -909.38   1818.8  0.0000      1
```

```
## ditchVSfield_veg_siteArea  8 1832.7 1859.5 -908.34   1816.7  2.0851      0
## ditchVSfield_veg_areaPlot  8 1834.0 1860.8 -908.99   1818.0  0.0000      0
## ditchVSfield_veg_full      9 1834.7 1864.8 -908.34   1816.7  1.3037      1
##                            Pr(>Chisq)
## ditchVSfield_veg_noRE
## ditchVSfield_veg_plotOnly     1.0000
## ditchVSfield_veg_siteOnly     <2e-16 ***
## ditchVSfield_veg_noArea       1.0000
## ditchVSfield_veg_siteArea     <2e-16 ***
## ditchVSfield_veg_areaPlot     1.0000
## ditchVSfield_veg_full         0.2535
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##siteArea is best
```

Check diagnostics

```
#simulate residuals
residuals_ditchVSfield_veg_siteArea <- simulateResiduals(fittedModel = ditchVSfield_veg_siteArea, plot =
```




```
##results indicate plot is not a good fit.
```

Try transforming data log(x+1) and running gaussian model

```r
##Log transform and add one to account for zeros.
surveys$log_grass_forb <- log(surveys$grass_forb_perc + 1)

##Model with gaussian distribution
ditchVSfield_veg_full <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|area/site_id/plot_id),
                            family = gaussian(),
                            data = surveys)
ditchVSfield_veg_noArea <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|site_id/plot_id),
                            family = gaussian(),
                            data = surveys)

ditchVSfield_veg_plotOnly <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|plot_id),
                            family = gaussian(),
                            data = surveys)

ditchVSfield_veg_siteOnly <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|site_id),
                            family = gaussian(),
                            data = surveys)
ditchVSfield_veg_siteArea <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|area/site_id),
                            family = gaussian(),
                            data = surveys)

ditchVSfield_veg_areaPlot <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled +
                              (1|area/plot_id),
                            family = gaussian(),
                            data = surveys)

ditchVSfield_veg_noRE <- glmmTMB(log_grass_forb ~ cover_type*plot_type + julian.date_scaled,
                            family = gaussian(),
                            data = surveys)

#compare models
anova(ditchVSfield_veg_full, ditchVSfield_veg_noArea, ditchVSfield_veg_plotOnly, ditchVSfield_veg_siteOn
      ditchVSfield_veg_areaPlot, ditchVSfield_veg_noRE)
```

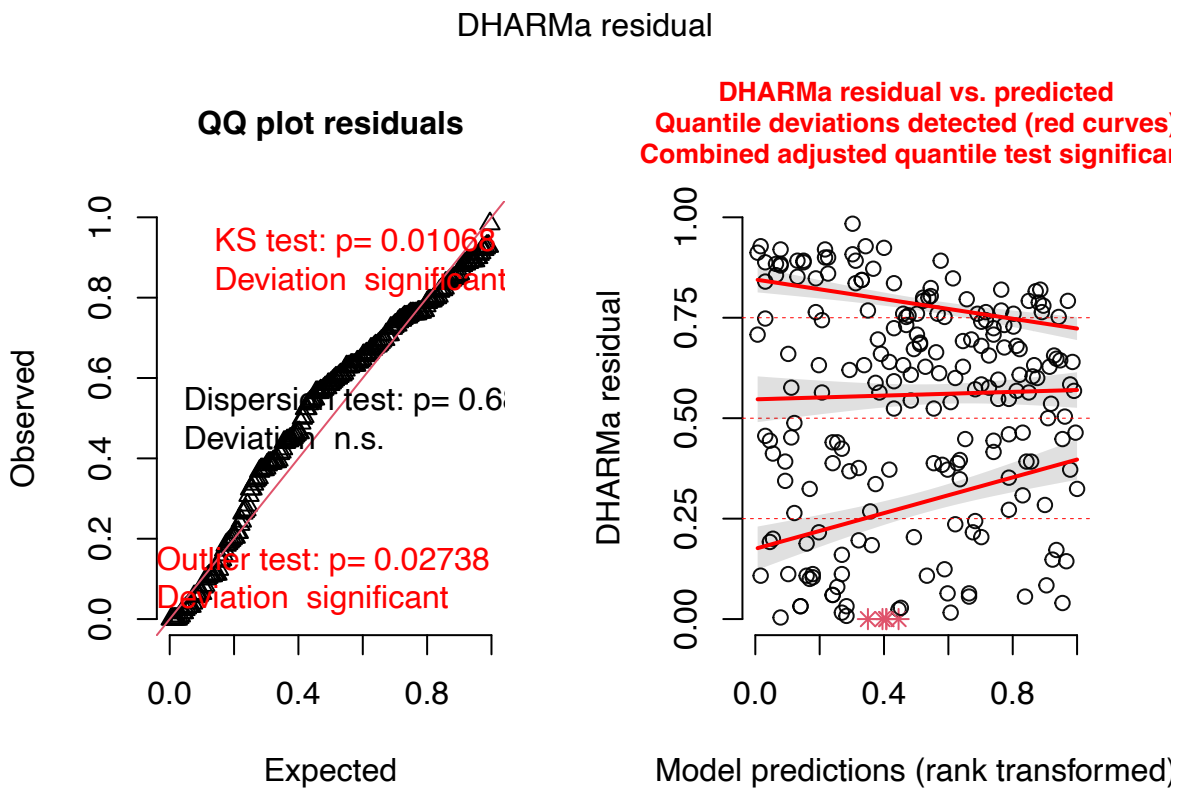
```
## Data: surveys
## Models:
## ditchVSfield_veg_noRE: log_grass_forb ~ cover_type * plot_type + julian.date_scaled, zi=~0, disp=~1
## ditchVSfield_veg_plotOnly: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, di
## ditchVSfield_veg_plotOnly:     (1 | plot_id), zi=~0, disp=~1
## ditchVSfield_veg_siteOnly: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, di
## ditchVSfield_veg_siteOnly:     (1 | site_id), zi=~0, disp=~1
## ditchVSfield_veg_noArea: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, disp
## ditchVSfield_veg_noArea:     (1 | site_id/plot_id), zi=~0, disp=~1
## ditchVSfield_veg_siteArea: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, di
## ditchVSfield_veg_siteArea:     (1 | area/site_id), zi=~0, disp=~1
## ditchVSfield_veg_areaPlot: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, di
## ditchVSfield_veg_areaPlot:     (1 | area/plot_id), zi=~0, disp=~1
```

```
## ditchVSfield_veg_full: log_grass_forb ~ cover_type * plot_type + julian.date_scaled + , zi=~0, disp=
## ditchVSfield_veg_full:     (1 | area/site_id/plot_id), zi=~0, disp=~1
##                           Df    AIC    BIC  logLik deviance   Chisq Chi Df
## ditchVSfield_veg_noRE      6 548.54 568.62 -268.27   536.54
## ditchVSfield_veg_plotOnly  7 550.54 573.97 -268.27   536.54  0.0000      1
## ditchVSfield_veg_siteOnly  7 507.68 531.11 -246.84   493.68 42.8551      0
## ditchVSfield_veg_noArea    8 509.68 536.46 -246.84   493.68  0.0000      1
## ditchVSfield_veg_siteArea  8 506.91 533.69 -245.46   490.91  2.7715      0
## ditchVSfield_veg_areaPlot  8 511.42 538.20 -247.71   495.42  0.0000      0
## ditchVSfield_veg_full      9 508.91 539.03 -245.46   490.91  4.5149      1
##                           Pr(>Chisq)
## ditchVSfield_veg_noRE
## ditchVSfield_veg_plotOnly     1.0000
## ditchVSfield_veg_siteOnly     <2e-16 ***
## ditchVSfield_veg_noArea       1.0000
## ditchVSfield_veg_siteArea     <2e-16 ***
## ditchVSfield_veg_areaPlot     1.0000
## ditchVSfield_veg_full         0.0336 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##siteArea is best
```

Check diagnostics

```
#simulate residuals
residuals_ditchVSfield_veg_siteArea <- simulateResiduals(fittedModel = ditchVSfield_veg_siteArea, plot =
```




```
##model fit is good enough.
```

Print results

```
summary(ditchVSfield_veg_siteArea)
```

```
##  Family: gaussian  ( identity )
## Formula:
## log_grass_forb ~ cover_type * plot_type + julian.date_scaled +
##     (1 | area/site_id)
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     506.9     533.7    -245.5     490.9       202
##
## Random effects:
##
## Conditional model:
##  Groups       Name        Variance Std.Dev.
##  site_id:area (Intercept) 0.06077  0.2465
##  area         (Intercept) 0.14504  0.3808
##  Residual                 0.54745  0.7399
## Number of obs: 210, groups:  site_id:area, 12; area, 6
##
## Dispersion estimate for gaussian family (sigma^2): 0.547
```

```
##
## Conditional model:
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                     3.43978    0.22353  15.388  < 2e-16 ***
## cover_typegrass                 0.47699    0.22714   2.100 0.035727 *
## plot_typefield                 -1.16027    0.15317  -7.575 3.59e-14 ***
## julian.date_scaled             -0.08073    0.05165  -1.563 0.118033
## cover_typegrass:plot_typefield  0.76598    0.21662   3.536 0.000406 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Find effect of cover type within each plot type

```
library(emmeans)

pairs(emmeans(ditchVSfield_veg_siteArea, ~ cover_type | plot_type))
```

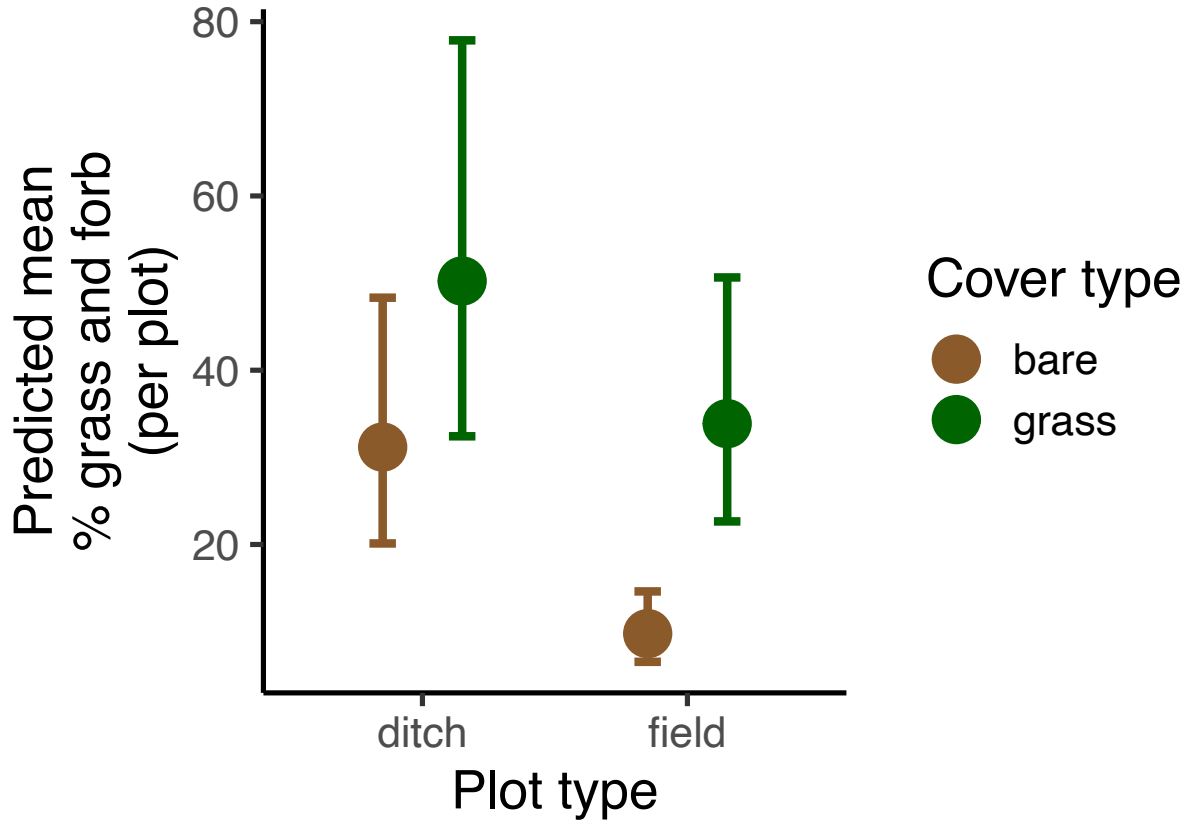
```
## plot_type = ditch:
##  contrast     estimate    SE  df t.ratio p.value
##  bare - grass   -0.477 0.227 202  -2.100  0.0370
##
## plot_type = field:
##  contrast     estimate    SE  df t.ratio p.value
##  bare - grass   -1.243 0.190 202  -6.556  <.0001
```

Plots

Cover type x plot type

```
#create new data grid
veg_data<- expand.grid(
  cover_type = unique(surveys$cover_type),
  plot_type = unique(surveys$plot_type),
  julian.date_scaled = mean(surveys$julian.date_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_veg <- predict(ditchVSfield_veg_siteArea,
                            newdata = veg_data,
                            se.fit = TRUE,
                            re.form = NA,
                           type = "link")

# Combine predictions with data frame
pred_veg_df <- cbind(veg_data,
                 fit_log = pred_veg$fit,
                 se_log = pred_veg$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_veg_df <- pred_veg_df %>%
  mutate(
    fit = exp(fit_log),
```

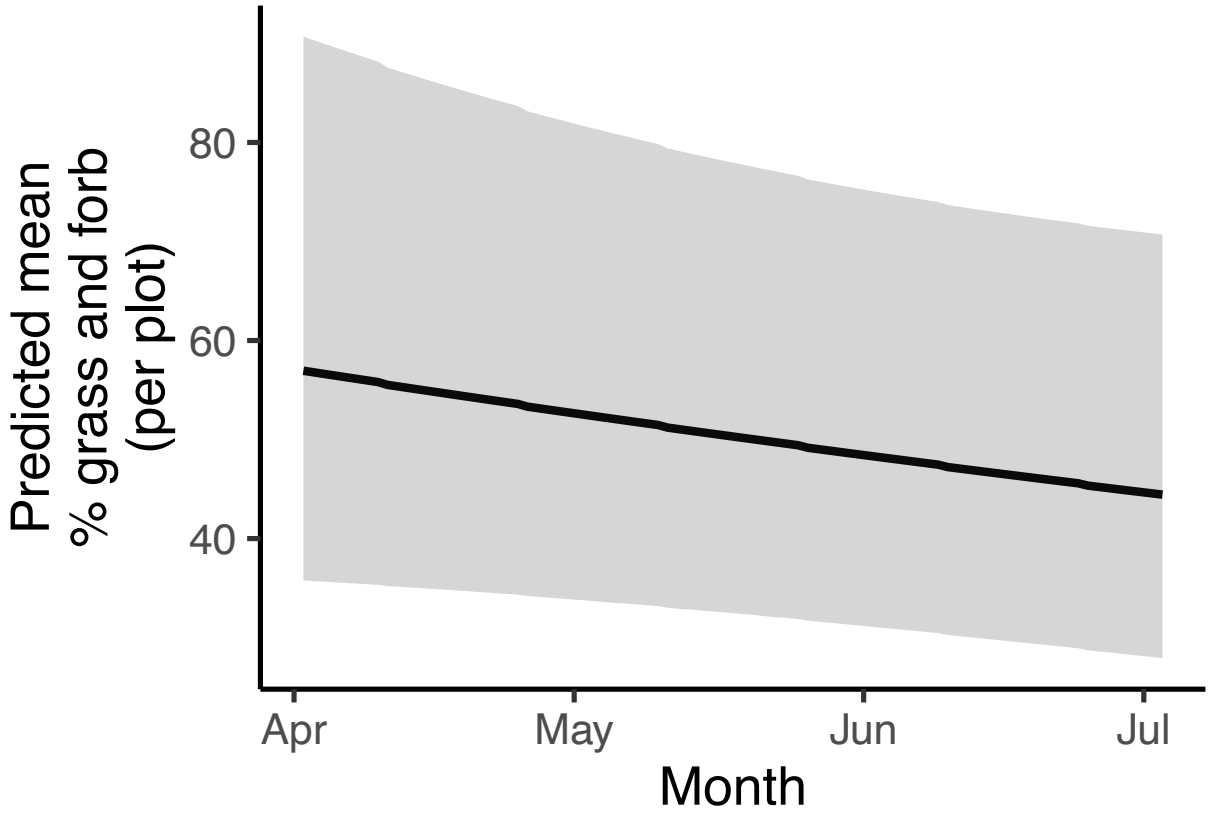
```
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_veg_df, aes(x = plot_type, y = fit, color = cover_type)) +
  geom_point(size=8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  # Use scale_color_manual to define specific colors
  scale_color_manual(values = c("grass" = "darkgreen", "bare" = "tan4")) +
  labs(x = "Plot type",
       y = "Predicted mean\n% grass and forb\n(per plot)",
       color = "Cover type") +
  theme_classic(base_size = 20)
```



Julian date

```
date_seq <- seq(
  from = min(surveys$julian.date_scaled, na.rm = TRUE),
  to = max(surveys$julian.date_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
veg_data_date<- expand.grid(
  cover_type = "grass",
```

```
  plot_type = "ditch",
  julian.date_scaled = date_seq
)

# Get predicted probabilities and standard errors
pred_veg_date <- predict(ditchVSfield_veg_siteArea,
                         newdata = veg_data_date,
                         se.fit = TRUE,
                         re.form = NA,
                        type = "link")

# Combine predictions with data frame
pred_veg_date_df <- cbind(veg_data_date,
                fit_log = pred_veg_date$fit,
                se_log = pred_veg_date$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_veg_date_df <- pred_veg_date_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#convert x-axis to actual dates
## Get scaling parameters from original data
jd_mean <- mean(surveys$julian.date, na.rm = TRUE)
jd_sd <- sd(surveys$julian.date, na.rm = TRUE)

## Convert scaled values back to dates
pred_veg_date_df <- pred_veg_date_df %>%
  mutate(
    # Reverse the (x - mean) / sd formula
    julian_orig = (julian.date_scaled * jd_sd) + jd_mean,
    # Convert Day of Year to an actual Date object (using 2024 as a reference year)
    date_formatted = as.Date(round(julian_orig) - 1, origin = "2024-01-01")
  )

#Plot
ggplot(pred_veg_date_df, aes(x = date_formatted, y = fit)) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2, color=NA) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") + # Format x-axis to show Month names (e.g.
  labs(x = "Month",
       y = "Predicted mean\n% grass and forb\n(per plot)") +
  theme_classic(base_size = 20)
```



Findings

Cover type

- Significantly more veg at grassy sites than bare both along the ditch (p=0.037) and within the field (p<0.0001)
- Significant interaction between plot type and cover type, difference in veg between grassy and bare is much greater in the field than along the ditch (p=0.0004)
- Significantly less veg within the field than along the ditch (p<0.0001)

Date

- % veg decreases over time, though not significant (p=0.118)

Floral

Steps

Try different random effects structures

```
#Load packages needed for CLM
library(ordinal)
```

```
##
## Attaching package: 'ordinal'
```

```
## The following object is masked from 'package:dplyr':
##
##     slice
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
surveys$bombus_floral_abun <- factor(surveys$bombus_floral_abun, ordered = TRUE)

floral_full <- clmm(bombus_floral_abun ~ cover_type*plot_type +
                        julian.date_scaled*plot_type +
                        (1|area/site_id/plot_id),
                      link = "logit",
                      data = surveys)

floral_noArea <- clmm(bombus_floral_abun ~ cover_type*plot_type +
                        julian.date_scaled*plot_type +
                        (1|site_id/plot_id),
                      link = "logit",
                      data = surveys)

#floral_plotOnly <- clmm(bombus_floral_abun ~ plot_type +
 #                         julian.date_scaled +
  #                        (1|plot_id),
   #                     link = "logit",
    #                    data = surveys)
##can't use since there are only 2 levels

floral_siteOnly <- clmm(bombus_floral_abun ~ cover_type*plot_type +
                        julian.date_scaled*plot_type +
                        (1|site_id),
                      link = "logit",
                      data = surveys)

floral_AreaPlot <- clmm(bombus_floral_abun ~ cover_type*plot_type +
                        julian.date_scaled*plot_type +
                        (1|area/plot_id),
                      link = "logit",
                      data = surveys)
```

```
floral_noRE <- polr(bombus_floral_abun ~ cover_type*plot_type +
                        julian.date_scaled*plot_type,
                      method = "logistic",
                      data = surveys)

#compare models AIC
AIC(floral_full, floral_noArea, floral_siteOnly, floral_AreaPlot, floral_noRE)
```

```
##                 df      AIC
## floral_full     12 658.1918
## floral_noArea   11 656.1918
## floral_siteOnly 10 654.1920
## floral_AreaPlot 11 656.9131
## floral_noRE      9 652.9131
```

```
##shows that model without random effect has best fit
```

Print results

```
summary(floral_noRE)
```

```
##
## Re-fitting to get Hessian

## Call:
## polr(formula = bombus_floral_abun ~ cover_type * plot_type +
##     julian.date_scaled * plot_type, data = surveys, method = "logistic")
##
## Coefficients:
##                                    Value Std. Error t value
## cover_typegrass                   0.2150     0.4054  0.5305
## plot_typefield                    0.3689     0.3668  1.0057
## julian.date_scaled                0.6428     0.2066  3.1119
## cover_typegrass:plot_typefield   -0.1843     0.5126 -0.3596
## plot_typefield:julian.date_scaled -0.9020     0.2619 -3.4442
##
## Intercepts:
##     Value   Std. Error t value
## 0|1 -1.2577  0.3169    -3.9687
## 1|2 -0.7560  0.3051    -2.4778
## 2|3  0.1283  0.2987     0.4294
## 3|4  1.1876  0.3116     3.8114
##
## Residual Deviance: 634.9131
## AIC: 652.9131
```

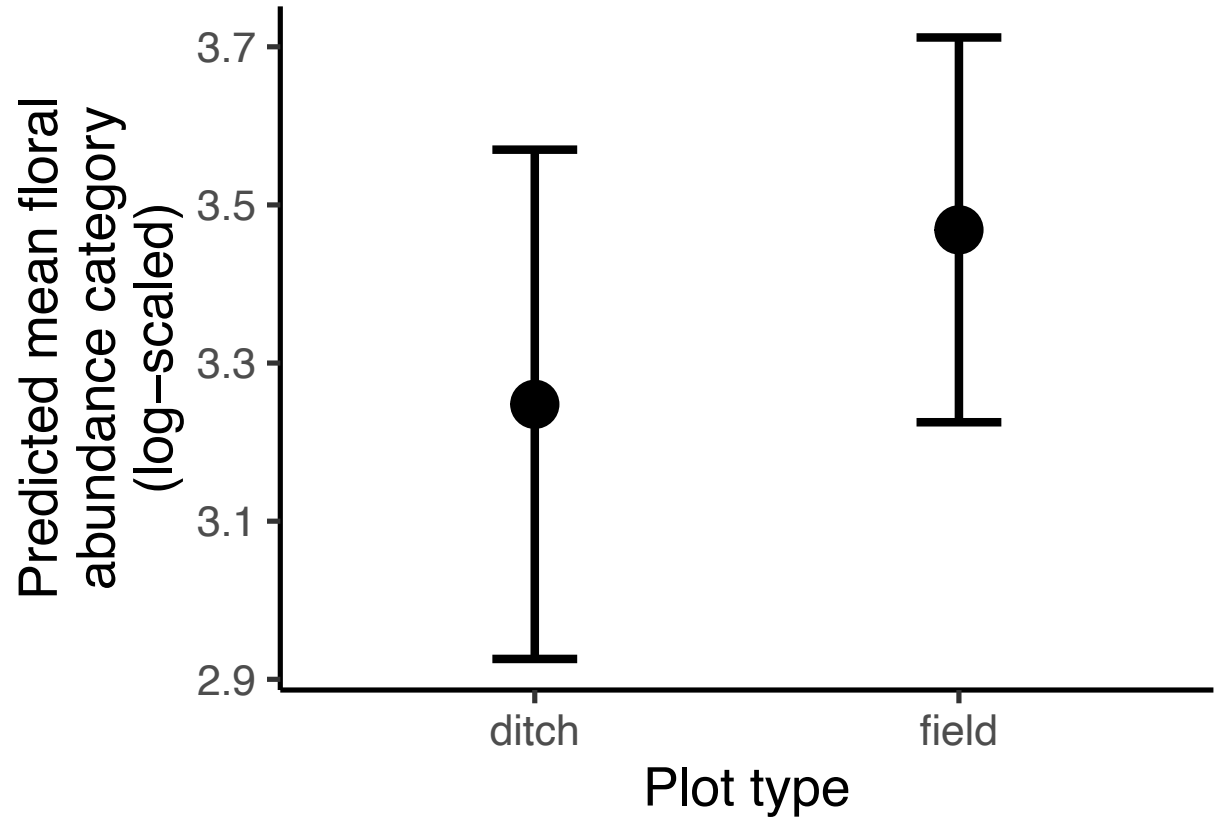
Plots

Plot type

```r
# Get predicted probabilities per category with CIs
pred_floral_plot <- emmeans(
  floral_noRE,
  ~ plot_type,
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)
```

```
##
## Re-fitting to get Hessian

## NOTE: Results may be misleading due to involvement in interactions
```

```r
# Convert to dataframe
pred_floral_plot_df <- as.data.frame(pred_floral_plot)

#plot
ggplot(pred_floral_plot_df, aes(y = mean.class, x = plot_type)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.2, linewidth = 1.5, position = posit
  labs(
    x = "Plot type",
    y = "Predicted mean floral\nabundance category\n(log-scaled)"
  ) +
  theme_classic(base_size = 20)
```

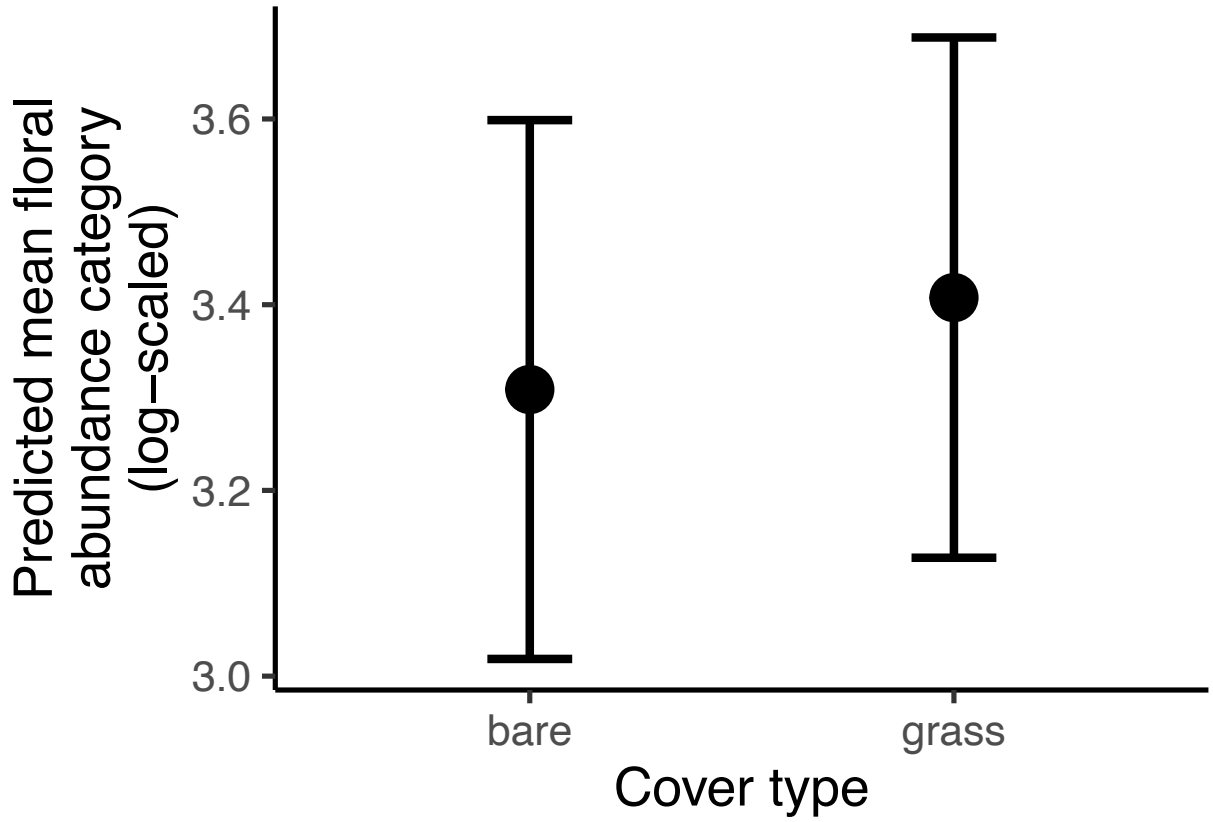


Cover type

```r
# Get predicted probabilities per category with CIs
pred_floral_cover <- emmeans(
  floral_noRE,
  ~ cover_type,
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)
```

```
##
## Re-fitting to get Hessian

## NOTE: Results may be misleading due to involvement in interactions
```

```r
# Convert to dataframe
pred_floral_cover_df <- as.data.frame(pred_floral_cover)

#plot
ggplot(pred_floral_cover_df, aes(x = cover_type, y = mean.class)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.2, linewidth = 1.5, position = posit
  labs(
    x = "Cover type",
    y = "Predicted mean floral\nabundance category\n(log-scaled)"
  ) +
  theme_classic(base_size = 20)
```



Season x plot type

```r
# Sequence of Julian dates
date_seq <- seq(
  from = min(surveys$julian.date_scaled, na.rm = TRUE),
  to = max(surveys$julian.date_scaled, na.rm = TRUE),
  length.out = 200
)

# Get predicted probabilities per category with CIs
pred_floral_date <- emmeans(
  floral_noRE,
  ~ julian.date_scaled * plot_type,
  at = list(julian.date_scaled = date_seq),
  mode = "mean.class" ## The expected floral abundance category (mean ordinal score)
)
```

```
##
## Re-fitting to get Hessian
```

```r
# Convert to dataframe
pred_floral_date_df <- as.data.frame(pred_floral_date)

#convert x-axis to actual dates
## Convert scaled values back to dates
pred_floral_date_df <- pred_floral_date_df %>%
  mutate(
    # Reverse the (x - mean) / sd formula
    julian_orig = (julian.date_scaled * jd_sd) + jd_mean,
    # Convert Day of Year to an actual Date object (using 2024 as a reference year)
    date_formatted = as.Date(round(julian_orig) - 1, origin = "2024-01-01")
  )

min(pred_floral_date_df$date_formatted, na.rm = TRUE)
```

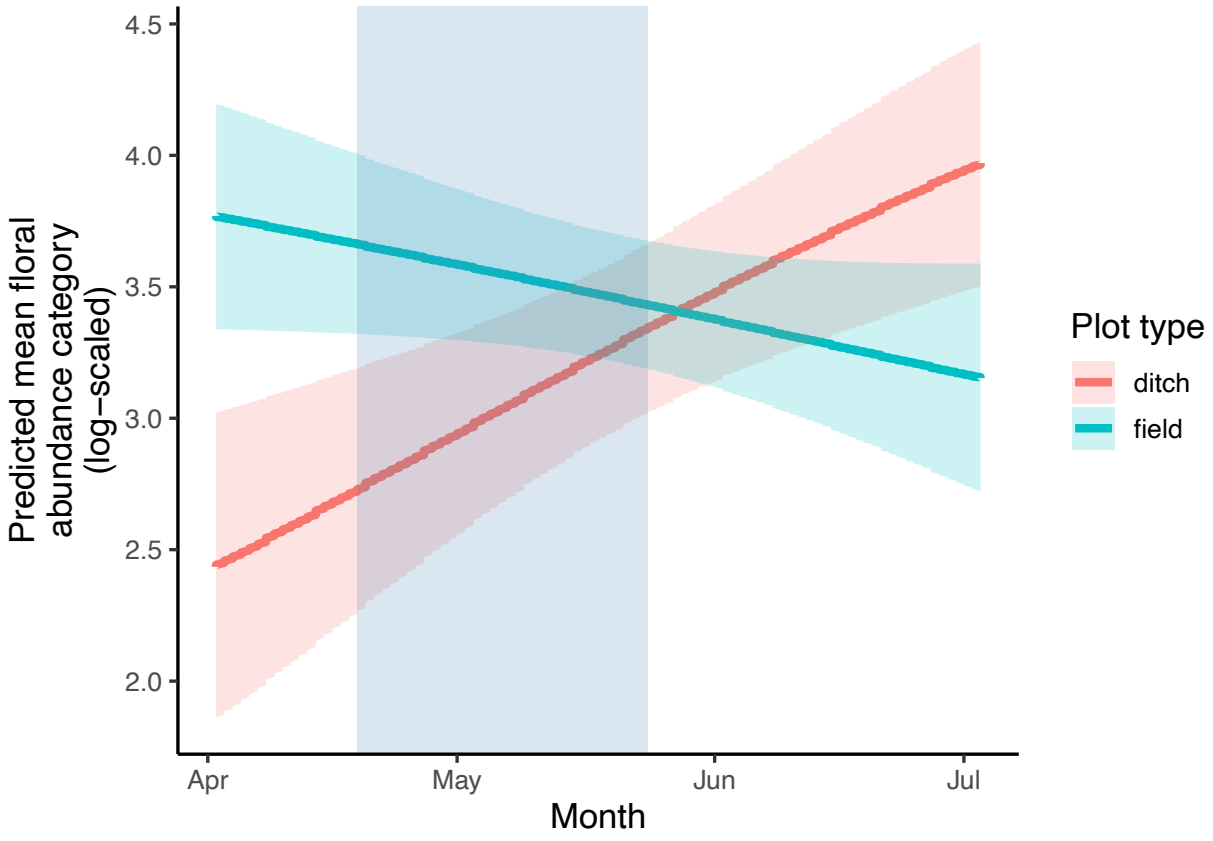
```
## [1] "2024-04-02"
```

```r
max(pred_floral_date_df$date_formatted, na.rm = TRUE)
```

```
## [1] "2024-07-03"
```

```r
#set peak bloom start and end dates
bloom_start <- as.Date("2024-04-19")
bloom_end <- as.Date("2024-05-24")

#plot
ggplot(pred_floral_date_df, aes(x = date_formatted, y = mean.class, color = plot_type)) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = asymp.LCL, ymax = asymp.UCL, fill = plot_type), alpha = 0.2, color = NA) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") + # Format x-axis to show Month names (e.g.
  annotate("rect", xmin = bloom_start, xmax = bloom_end, ymin = -Inf, ymax = Inf,
           alpha = .2, fill = "steelblue")+
  labs(
```

```
    x = "Month",
    y = "Predicted mean floral\nabundance category\n(log-scaled)",
    color = "Plot type",
    fill = "Plot type"
  ) +
  theme_classic(base_size = 13)
```



Findings

Cover type

- No significant difference in floral abundance between ditch and field at beginning of the study (t=1.01)
- No difference in floral abundance between grass and bare sites (t=0.531).
- No significant interaction between plot type and cover type (t=-0.360)

Date

- Significant effect of julian date (t=3.112)
- Significant interaction between julian date and plot type (t=-3.444).
  - Ditches get more flowers over time (+0.64 per unit of time).
  - Fields get fewer flowers over time (-0.26 per unit of time)

Rodents

Steps

1) Try random effect structures with poisson.

- model with site only is best

```
rodent_veg_areaOnly <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 (1|area),
                               family = poisson,
                               data = surveys)

rodent_veg_siteOnly <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 (1|site_id),
                               family = poisson,
                               data = surveys)
rodent_veg_siteArea <- glmmTMB(num_burrows ~ cover_type*plot_type +
                                 (1|area/site_id),
                               family = poisson,
                               data = surveys)

#compare models
anova(rodent_veg_areaOnly, rodent_veg_siteOnly, rodent_veg_siteArea)
```

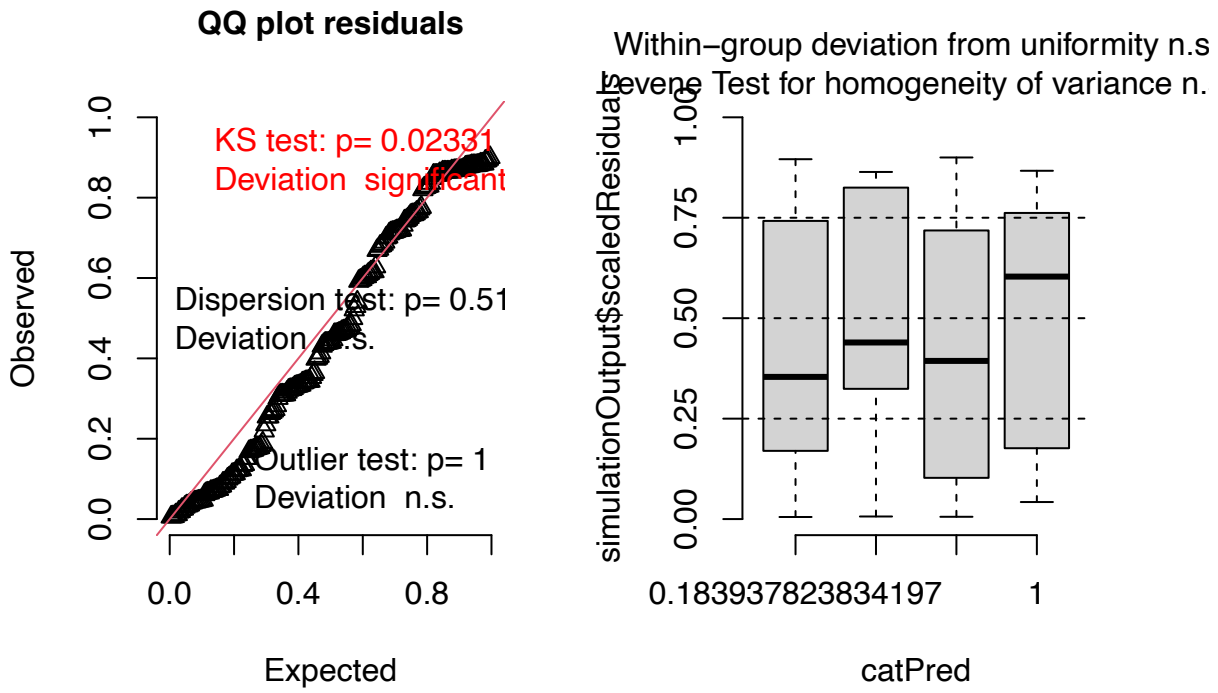
```
## Data: surveys
## Models:
## rodent_veg_areaOnly: num_burrows ~ cover_type * plot_type + (1 | area), zi=~0, disp=~1
## rodent_veg_siteOnly: num_burrows ~ cover_type * plot_type + (1 | site_id), zi=~0, disp=~1
## rodent_veg_siteArea: num_burrows ~ cover_type * plot_type + (1 | area/site_id), zi=~0, disp=~1
##                     Df    AIC    BIC  logLik deviance     Chisq Chi Df
## rodent_veg_areaOnly  5 4471.5 4488.3 -2230.8   4461.5
## rodent_veg_siteOnly  5 3390.2 3406.9 -1690.1   3380.2 1081.3478      0
## rodent_veg_siteArea  6 3392.1 3412.2 -1690.0   3380.1    0.1089      1
##                     Pr(>Chisq)
## rodent_veg_areaOnly
## rodent_veg_siteOnly     <2e-16 ***
## rodent_veg_siteArea     0.7414
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##site only is best
```

2) Check diagnostics

- KS test shows significant deviation

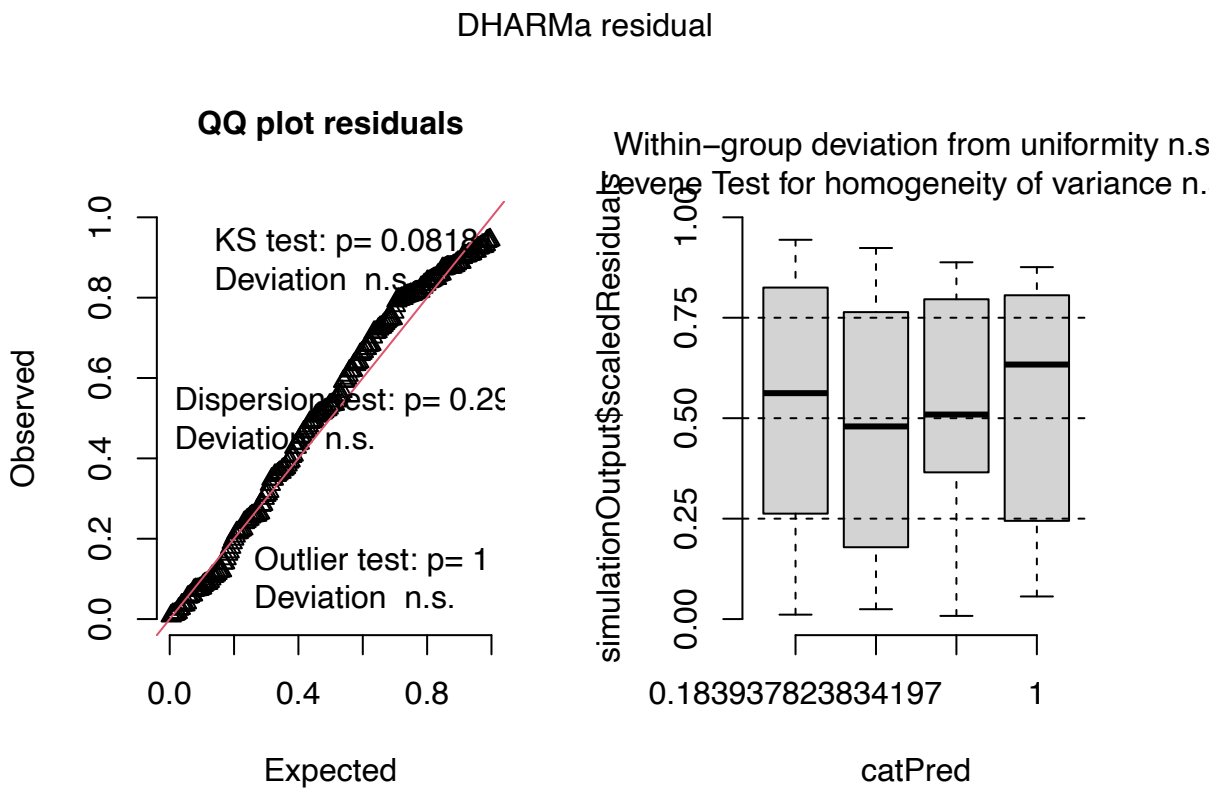
```
residuals_rodent_veg_siteOnly <- simulateResiduals(fittedModel = rodent_veg_siteOnly, plot = TRUE)
```



3) Try negative binomial and check diagnostics.

- diagnostic plots look good!

```
rodent_veg_siteOnly_nbin <- glmmTMB(num_burrows ~ cover_type*plot_type +
                              (1|site_id),
                            family = nbinom2,
                            data = surveys)

#check diagnostics
#simulate residuals
residuals_rodent_veg_siteOnly_nbin <- simulateResiduals(fittedModel = rodent_veg_siteOnly_nbin, plot = '
```



4. Plot results.

```
summary(rodent_veg_siteOnly_nbin)
```

```
##  Family: nbinom2  ( log )
## Formula:          num_burrows ~ cover_type * plot_type + (1 | site_id)
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    1485.0    1505.0    -736.5    1473.0       204
##
## Random effects:
##
## Conditional model:
##  Groups  Name        Variance Std.Dev.
##  site_id (Intercept) 2.681    1.637
## Number of obs: 210, groups:  site_id, 12
##
## Dispersion parameter for nbinom2 family (): 1.78
##
## Conditional model:
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                      3.0370     0.6866   4.423 9.72e-06 ***
## cover_typegrass                  0.2133     0.9699   0.220    0.826
## plot_typefield                  -2.0235     0.2097  -9.650  < 2e-16 ***
```

```
## cover_typegrass:plot_typefield   1.5099     0.2811   5.372 7.77e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Find effect of cover type in each plot type

```
pairs(emmeans(rodent_veg_siteOnly_nbin, ~ cover_type | plot_type))
```

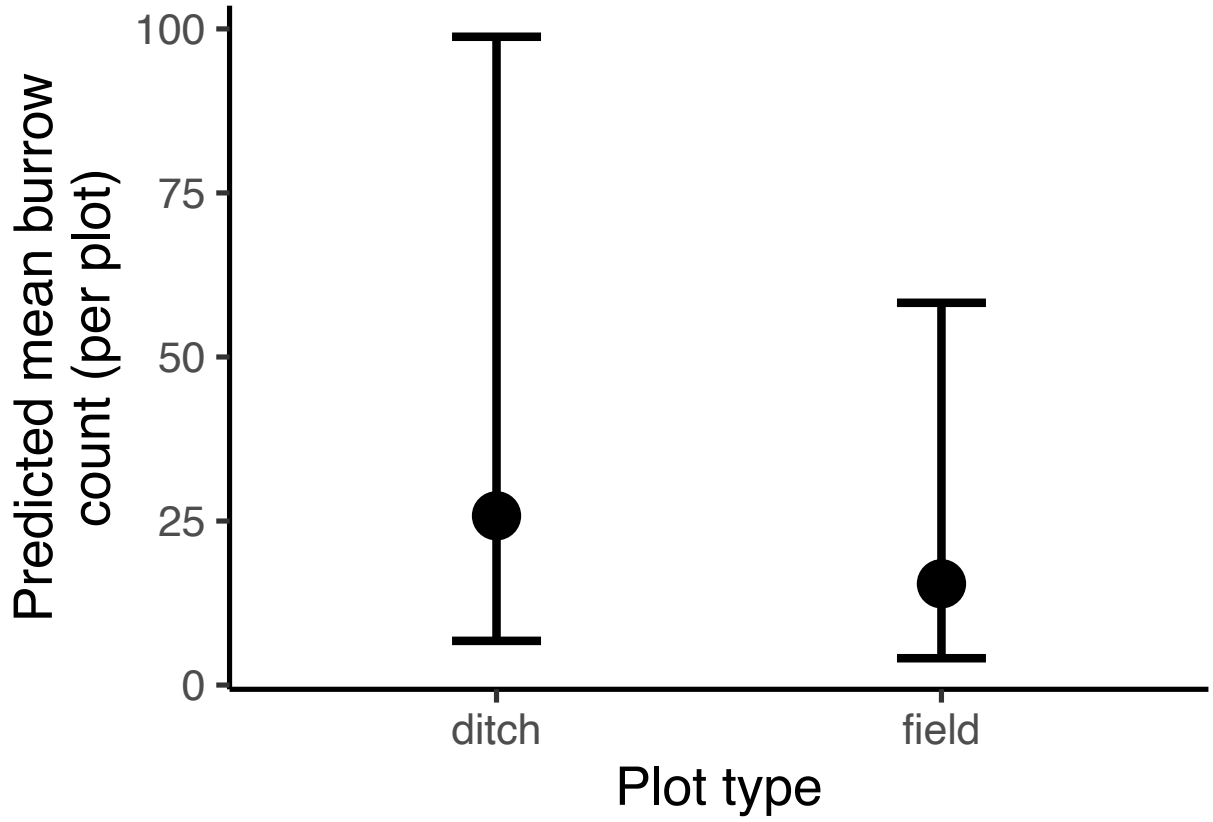
```
## plot_type = ditch:
##  contrast     estimate    SE  df z.ratio p.value
##  bare - grass   -0.213 0.970 Inf  -0.220  0.8259
##
## plot_type = field:
##  contrast     estimate    SE  df z.ratio p.value
##  bare - grass   -1.723 0.963 Inf  -1.790  0.0735
##
## Results are given on the log (not the response) scale.
```

Plots

Plot type

```
#create new data grid
rodent_data_plot<- expand.grid(
  cover_type = "grass",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_plot <- predict(rodent_veg_siteOnly_nbin,
                            newdata = rodent_data_plot,
                            se.fit = TRUE,
                            re.form = NA,
                           type = "link")

# Combine predictions with data frame
pred_rodent_plot_df <- cbind(rodent_data_plot,
                 fit_log = pred_rodent_plot$fit,
                 se_log = pred_rodent_plot$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_plot_df <- pred_rodent_plot_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_rodent_plot_df, aes(x = plot_type, y = fit)) +
  geom_point(size=8, position = position_dodge(0.6)) +
```

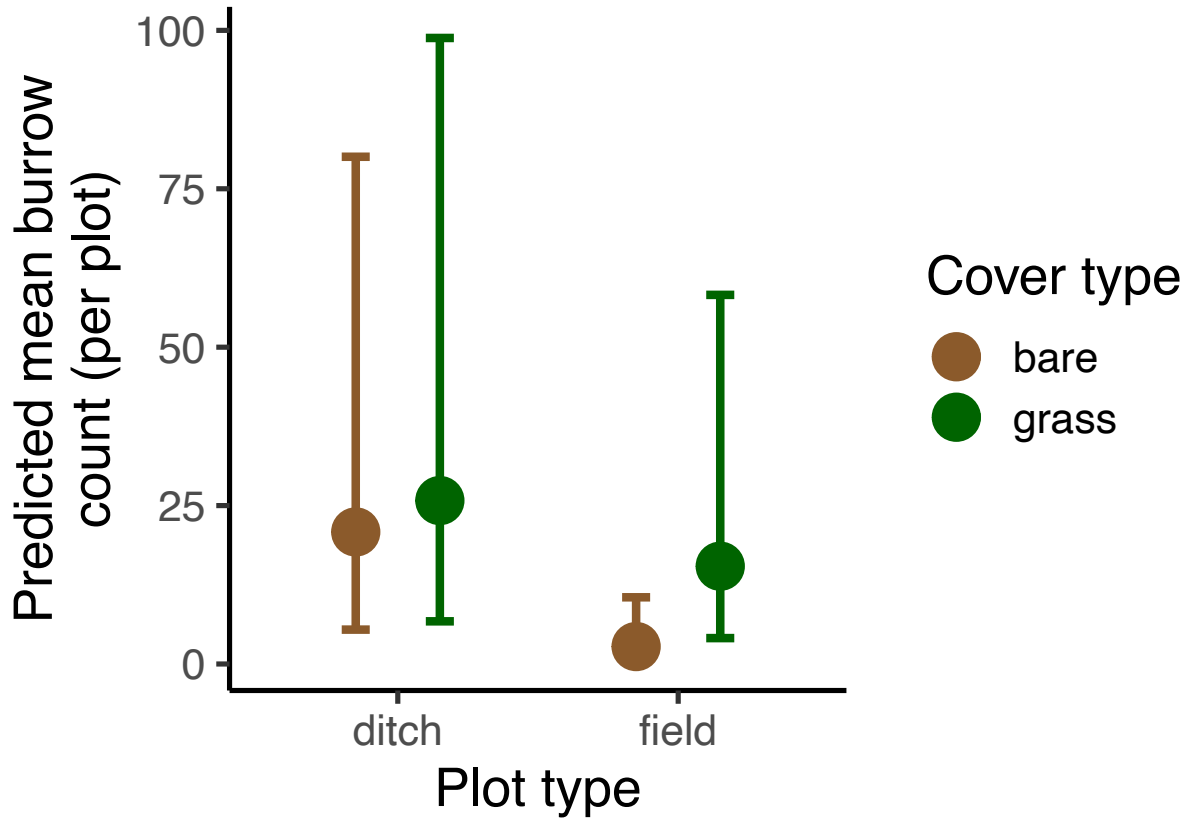
```
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Plot type",
       y = "Predicted mean burrow\ncount (per plot)") +
  theme_classic(base_size = 20)
```



cover type x plot type

```
#create new data grid
rodent_data_veg<- expand.grid(
  cover_type = unique(surveys$cover_type),
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_veg <- predict(rodent_veg_siteOnly_nbin,
                           newdata = rodent_data_veg,
                           se.fit = TRUE,
                           re.form = NA,
                          type = "link")

# Combine predictions with data frame
pred_rodent_veg_df <- cbind(rodent_data_veg,
                 fit_log = pred_rodent_veg$fit,
                 se_log = pred_rodent_veg$se.fit)
```

```r
#calculate predictions and confidence intervals on response scale (not log scale)
pred_rodent_veg_df <- pred_rodent_veg_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

# Plot conditional predictions
ggplot(pred_rodent_veg_df, aes(x = plot_type, y = fit, color = cover_type)) +
  geom_point(size=8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Plot type",
       y = "Predicted mean burrow\ncount (per plot)",
       color = "Cover type") +
  scale_color_manual(values = c("grass" = "darkgreen", "bare" = "tan4")) +
  theme_classic(base_size = 20)
```



Findings

- Significant interaction between cover type and plot type ($p<0.0001$)
  - Marginal effect of cover type on rodent burrow abundance within the field only ($p=0.0735$)
  - No effect of cover type on burrow abundance along the ditch (probably because less difference in vegetation) ($p=0.826$)

- Significantly more rodents along the ditch than within the field ($p<0.0001$)

NSQ

Plot model with interaction between vegetation variables and plot type to investigate whether the effects of vegetation cover and floral abundance on queen abundance/presence change depending on plot type (ditch vs field).

Steps

1) Try different random effect structures.

- Model without random effect fit data the best.

```
nsq_glm_veg_full <- glmmTMB(nest_searching_queens ~ cover_type*plot_type+
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type +
                            (1|area/site_id/plot_id),
                           ziformula = ~cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type +
                            (1|area/site_id/plot_id),  # zero-inflation model
                           family = poisson,
                           data = surveys)

nsq_glm_veg_noArea <- glmmTMB(nest_searching_queens ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type+
                            bee_season_type+
                            (1|site_id/plot_id),
                           ziformula = ~cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type +
                            (1|site_id/plot_id),  # zero-inflation model
                           family = poisson,
                           data = surveys)

nsq_glm_veg_justSite <- glmmTMB(nest_searching_queens ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type+
                            (1|site_id),
                           ziformula = ~cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type+
                            bee_season_type +
                            (1|site_id),  # zero-inflation model
                           family = poisson,
                           data = surveys)

nsq_glm_veg_noRE <- glmmTMB(nest_searching_queens ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type,
                           ziformula = ~cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
```

```
                              bee_season_type,  # zero-inflation model
                            family = poisson,
                            data = surveys)

#compare models
anova(nsq_glm_veg_noRE, nsq_glm_veg_justSite, nsq_glm_veg_noArea, nsq_glm_veg_full)
```

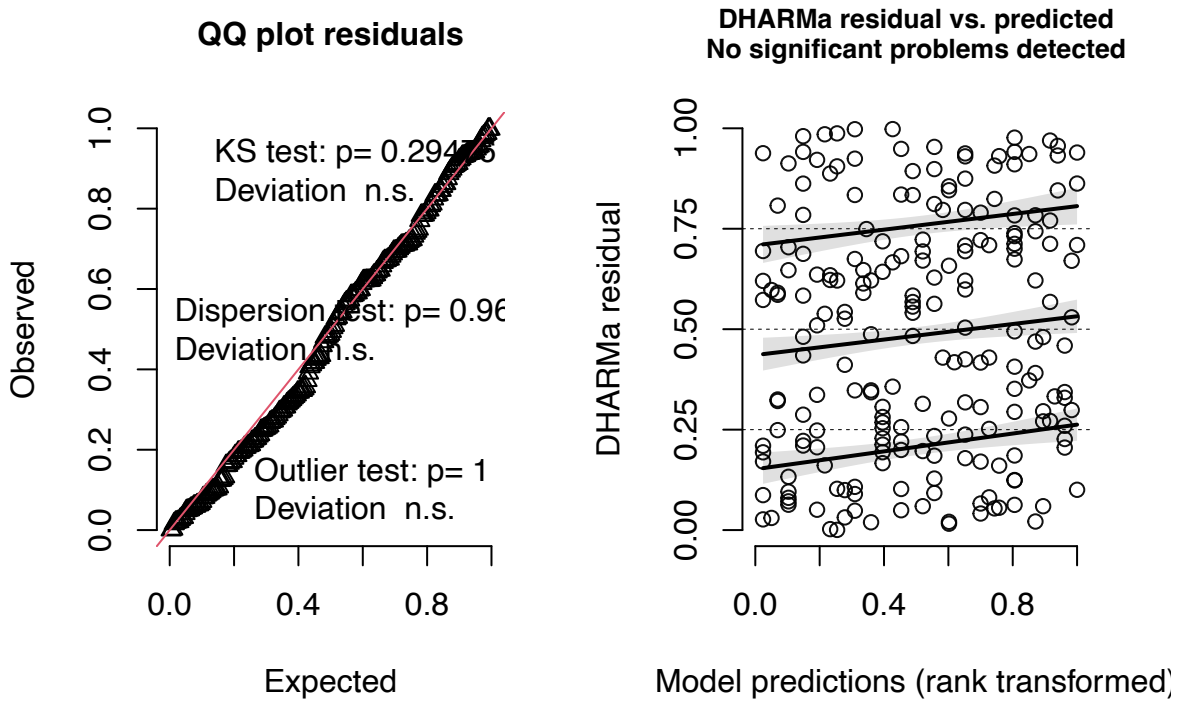
```
## Data: surveys
## Models:
## nsq_glm_veg_noRE: nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=
## nsq_glm_veg_noRE:     plot_type + bee_season_type, zi=    bee_season_type, disp=~1
## nsq_glm_veg_justSite: nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled * ,
## nsq_glm_veg_justSite:     plot_type + bee_season_type + (1 | site_id), zi=    bee_season_type + (1 |
## nsq_glm_veg_noArea: nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi
## nsq_glm_veg_noArea:     plot_type + bee_season_type + (1 | site_id/plot_id), zi=    bee_season_type
## nsq_glm_veg_full: nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=
## nsq_glm_veg_full:     plot_type + bee_season_type + (1 | area/site_id/plot_id), zi=    bee_season_ty
##                       Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## nsq_glm_veg_noRE      14 291.79 338.65 -131.90   263.79
## nsq_glm_veg_justSite  16 294.46 348.01 -131.23   262.46 1.3375      2     0.5123
## nsq_glm_veg_noArea    18 297.86 358.11 -130.93   261.86 0.5924      2     0.7436
## nsq_glm_veg_full      20 301.12 368.06 -130.56   261.12 0.7411      2     0.6904
```

```
##no random effect model is best, followed by just site model
```

2) Check diagnostics of model

- look good

```
#simulate residuals
residuals_nsq_glm_veg_noRE <- simulateResiduals(fittedModel = nsq_glm_veg_noRE, plot = TRUE)
```



3) Plot results

```
summary(nsq_glm_veg_noRE)
```

```
##  Family: poisson  ( log )
## Formula:
## nest_searching_queens ~ cover_type * plot_type + bombus_floral_abun_scaled *
##     plot_type + bee_season_type
## Zero inflation:
## ~cover_type * plot_type + bombus_floral_abun_scaled * plot_type +
##     bee_season_type
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     291.8     338.7    -131.9     263.8       196
##
##
## Conditional model:
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                               0.93434    0.28572   3.270  0.00108
## cover_typegrass                          -0.02272    0.31835  -0.071  0.94311
## plot_typefield                           -0.90770    0.62236  -1.458  0.14471
## bombus_floral_abun_scaled                -0.12284    0.15190  -0.809  0.41871
## bee_season_typeworker                    -0.96569    0.69606  -1.387  0.16533
## cover_typegrass:plot_typefield           -0.02475    0.68210  -0.036  0.97106
```

```
## plot_typefield:bombus_floral_abun_scaled  0.11710    0.30535   0.384  0.70135
##
## (Intercept)                              **
## cover_typegrass
## plot_typefield
## bombus_floral_abun_scaled
## bee_season_typeworker
## cover_typegrass:plot_typefield
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                               0.37909    0.53222   0.712  0.47629
## cover_typegrass                          -1.72465    0.77538  -2.224  0.02613
## plot_typefield                            1.13241    0.89586   1.264  0.20622
## bombus_floral_abun_scaled                 0.08354    0.39870   0.210  0.83404
## bee_season_typeworker                     2.59898    0.91079   2.853  0.00432
## cover_typegrass:plot_typefield           -0.73642    1.31238  -0.561  0.57471
## plot_typefield:bombus_floral_abun_scaled -0.80900    0.64334  -1.258  0.20858
##
## (Intercept)
## cover_typegrass                          *
## plot_typefield
## bombus_floral_abun_scaled
## bee_season_typeworker                    **
## cover_typegrass:plot_typefield
## plot_typefield:bombus_floral_abun_scaled
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

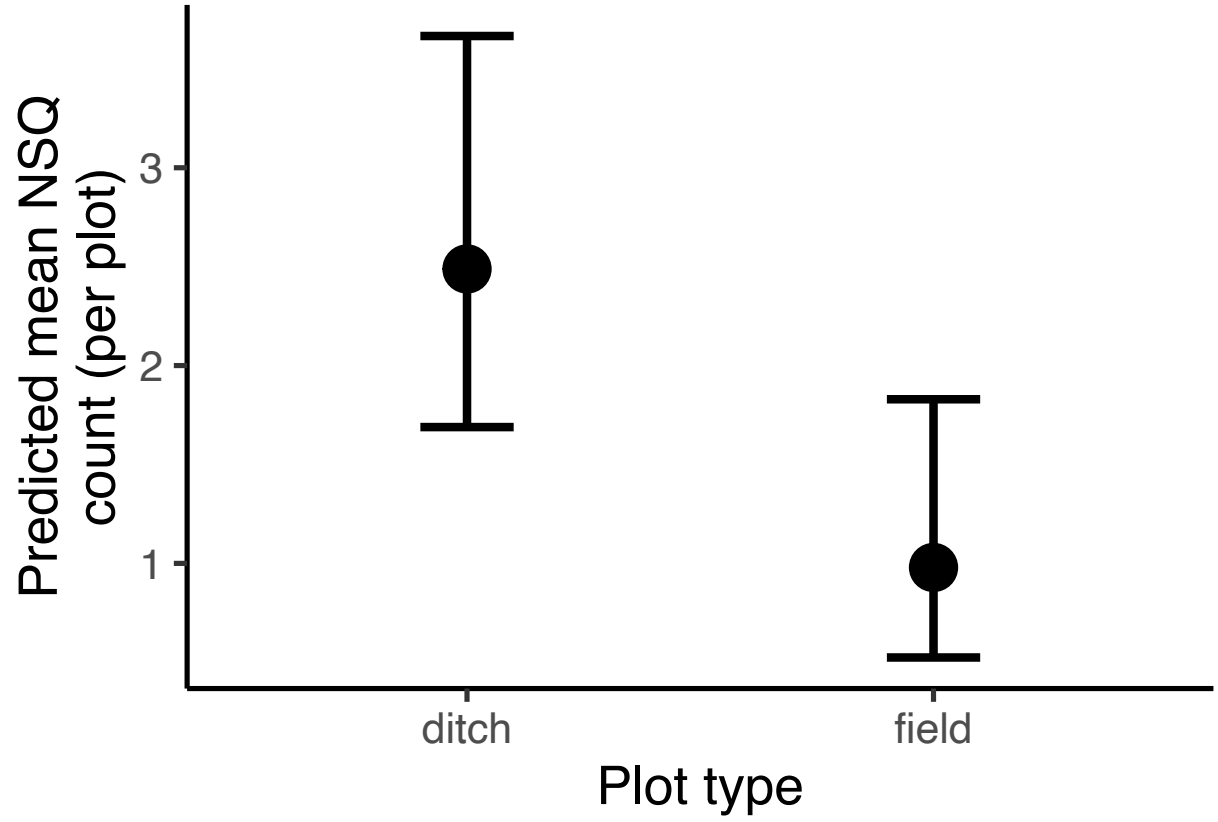
Plots

Plot type

Conditional

```
# Create newdata
nsq_data_plottype <- expand.grid(
  cover_type = "grass",
  plot_type = factor(c("ditch", "field"), levels = levels(surveys$plot_type))
)
nsq_data_plottype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
nsq_data_plottype$bee_season_type <- "nestsearching"


# Get predicted zero probabilities and standard errors
pred_nsq_plottype_cond <- predict(nsq_glm_veg_noRE,
                            newdata = nsq_data_plottype,
                            type = "link",
                            se.fit = TRUE)

# Combine predictions with data frame
```

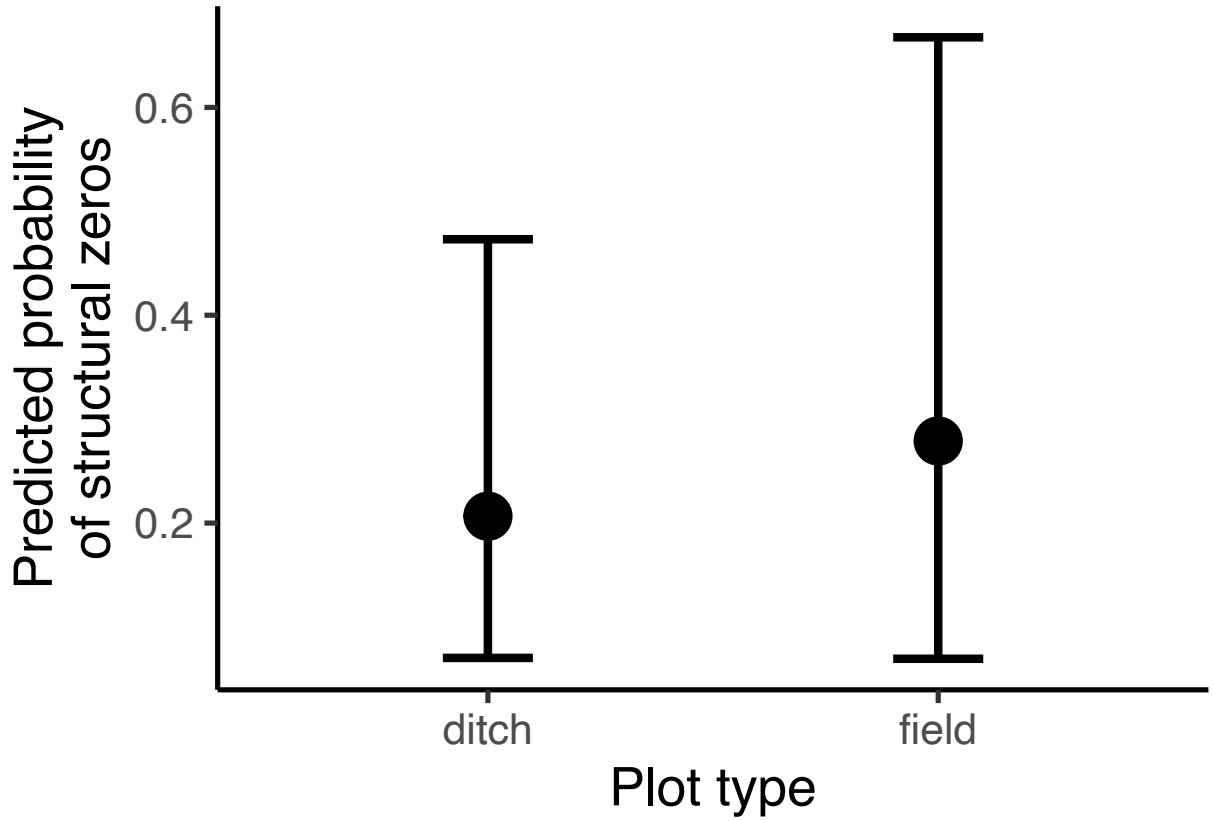
```r
pred_nsq_plottype_df <- cbind(nsq_data_plottype,
                 cond_fit_log = pred_nsq_plottype_cond$fit,
                 cond_se_log = pred_nsq_plottype_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_plottype_df <- pred_nsq_plottype_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#Plot
ggplot(pred_nsq_plottype_df, aes(x = plot_type, y = cond_fit)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Plot type",
       y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 20)
```



Zero-inflated model:

```r
# Predictions for ZI model
pred_nsq_plottype_zi <- predict(nsq_glm_veg_noRE,
                                newdata = nsq_data_plottype,
                                type = "zlink",
```

```
                                se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_plottype_zi_df <- cbind(
  nsq_data_plottype,
  zi_fit = plogis(pred_nsq_plottype_zi$fit),
  zi_lower = plogis(pred_nsq_plottype_zi$fit - 1.96 * pred_nsq_plottype_zi$se.fit),
  zi_upper = plogis(pred_nsq_plottype_zi$fit + 1.96 * pred_nsq_plottype_zi$se.fit)
)

# Plot zi predictions
ggplot(pred_nsq_plottype_zi_df, aes(x = plot_type, y = zi_fit)) +
  geom_point(size = 8,  position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = zi_lower, ymax = zi_upper), width = 0.2, position = position_dodge(0.6), lin
  labs(x = "Plot type",
       y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 20)
```

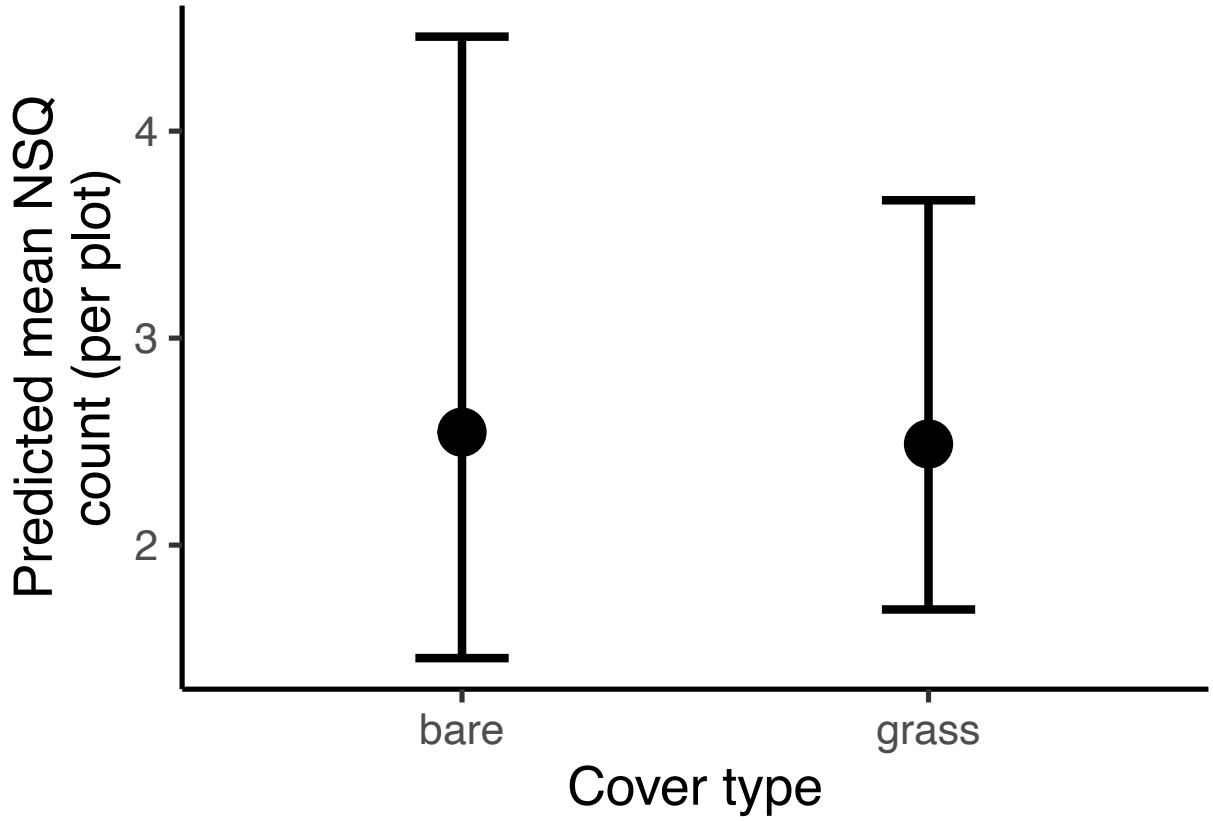


Cover type

Conditional

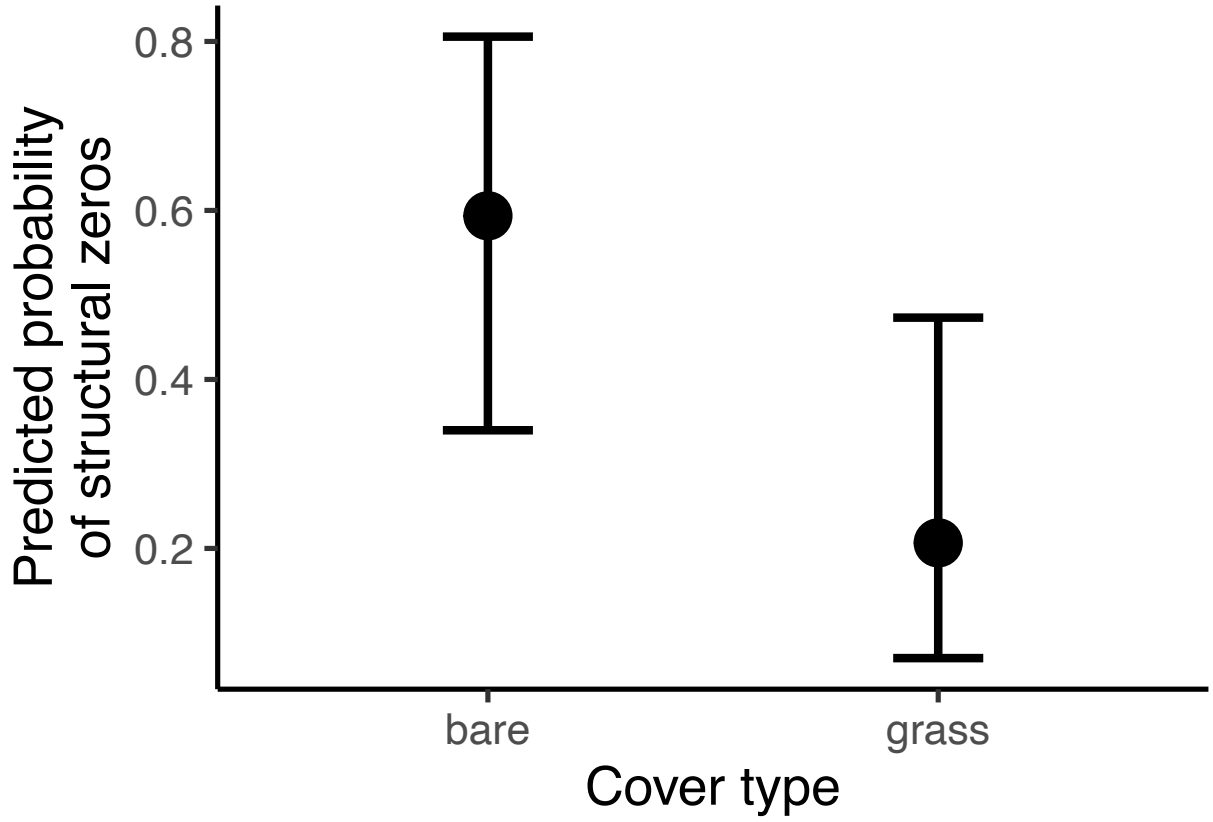
```
# Create newdata
nsq_data_covertype <- expand.grid(
  cover_type = factor(c("grass", "bare"), levels = levels(surveys$cover_type)),
  plot_type = "ditch"
```

```
)
nsq_data_covertype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
nsq_data_covertype$bee_season_type <- "nestsearching"


# Get predicted zero probabilities and standard errors
pred_nsq_covertype_cond <- predict(nsq_glm_veg_noRE,
                           newdata = nsq_data_covertype,
                           type = "link",
                           se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_covertype_df <- cbind(nsq_data_covertype,
                 cond_fit_log = pred_nsq_covertype_cond$fit,
                 cond_se_log = pred_nsq_covertype_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_covertype_df <- pred_nsq_covertype_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#Plot
ggplot(pred_nsq_covertype_df, aes(x = cover_type, y = cond_fit)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Cover type",
       y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 20)
```



Zero-inflated model:

```
# Predictions for ZI model
pred_nsq_covertype_zi <- predict(nsq_glm_veg_noRE,
                                 newdata = nsq_data_covertype,
                                 type = "zlink",
                                se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_covertype_zi_df <- cbind(
  nsq_data_covertype,
  zi_fit = plogis(pred_nsq_covertype_zi$fit),
  zi_lower = plogis(pred_nsq_covertype_zi$fit - 1.96 * pred_nsq_covertype_zi$se.fit),
  zi_upper = plogis(pred_nsq_covertype_zi$fit + 1.96 * pred_nsq_covertype_zi$se.fit)
)

# Plot zi predictions
ggplot(pred_nsq_covertype_zi_df, aes(x = cover_type, y = zi_fit)) +
  geom_point(size = 8,  position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = zi_lower, ymax = zi_upper), width = 0.2, position = position_dodge(0.6), lin
  labs(x = "Cover type",
       y = "Predicted probability\nof structural zeros") +
  theme_classic(base_size = 20)
```

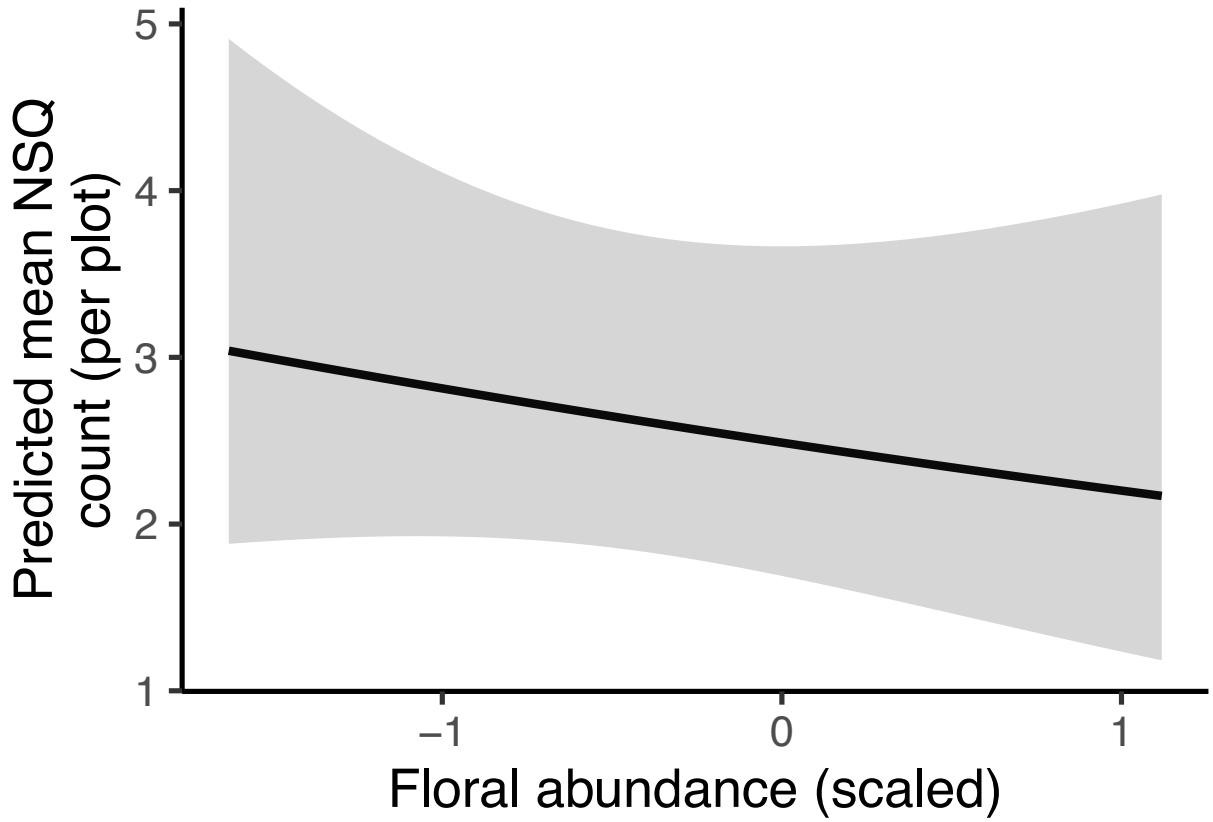


Floral abundance

Conditional model:

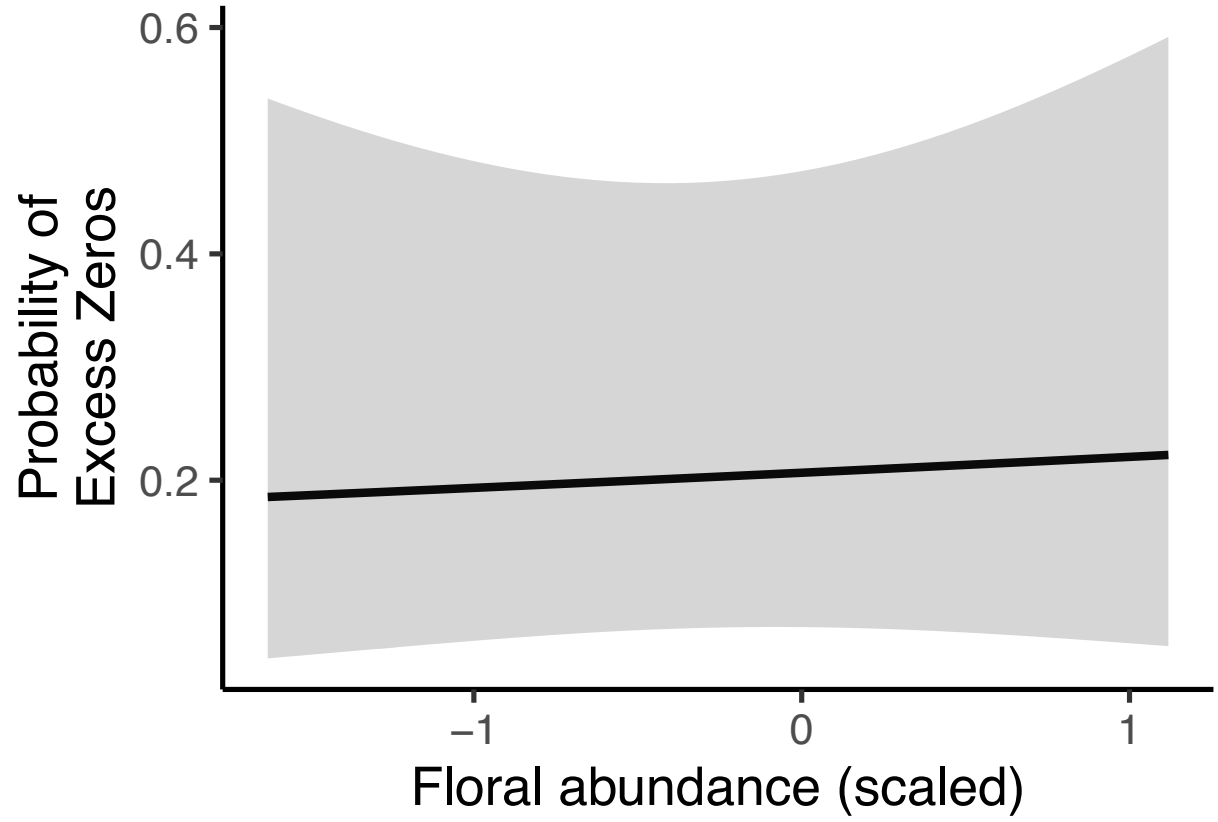
```
# Define a sequence of values for grass_forb_perc_scaled across its observed range
floral_seq <- seq(
  from = min(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  to = max(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
nsq_data_floral<- expand.grid(
  cover_type = "grass",
  plot_type = "ditch",
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "nestsearching"
)

# Get predicted conditional probabilities and standard errors
pred_nsq_floral_cond <- predict(nsq_glm_veg_noRE,
                            newdata = nsq_data_floral,
                            type = "link",
                            se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_floral_df <- cbind(nsq_data_floral,
```

```
                cond_fit_log = pred_nsq_floral_cond$fit,
                cond_se_log = pred_nsq_floral_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_floral_df <- pred_nsq_floral_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

#plot
ggplot(pred_nsq_floral_df,
       aes(x = bombus_floral_abun_scaled,
           y = cond_fit)) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = cond_lower, ymax = cond_upper),
              alpha = 0.2,
              color = NA) +
  labs(
    x = "Floral abundance (scaled)",
    y = "Predicted mean NSQ\ncount (per plot)"
  ) +
  theme_classic(base_size = 20)
```



Zero-inflated model:

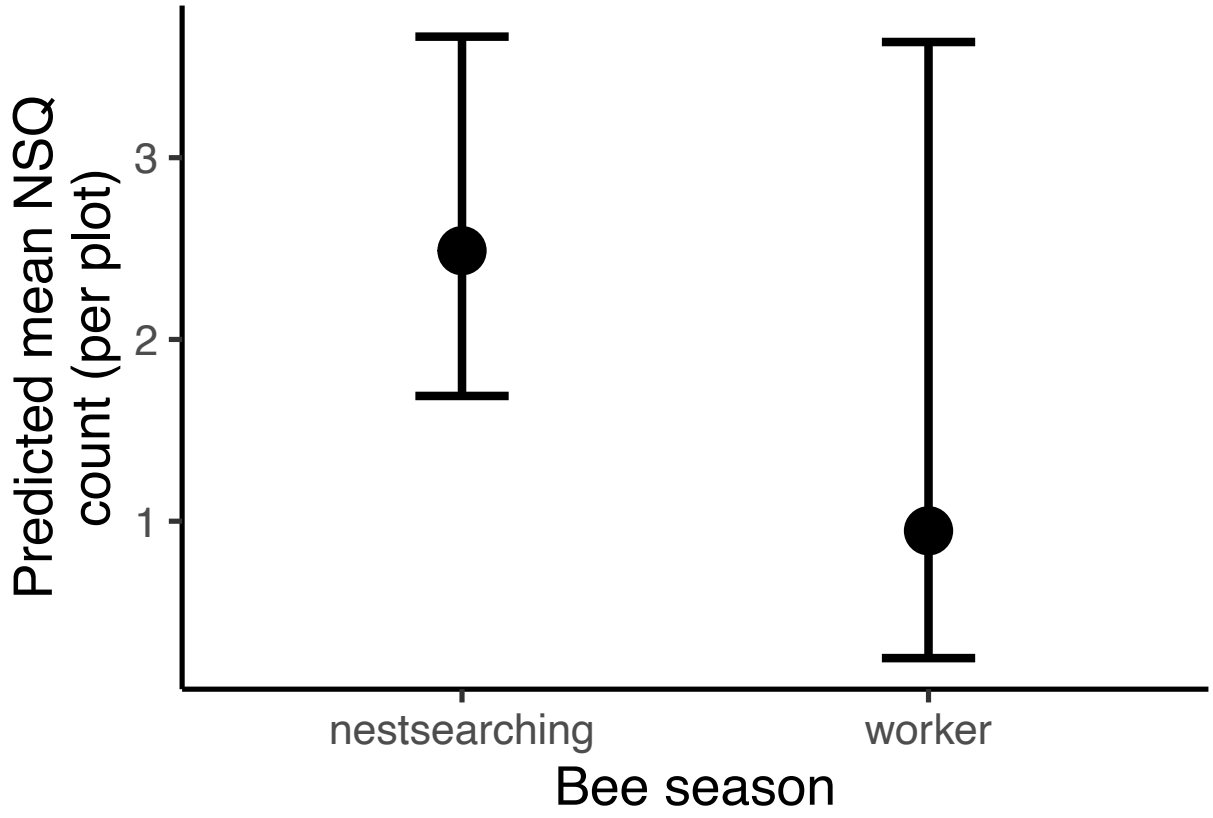
```r
# Get predicted zero probabilities and standard errors
pred_nsq_floral_zi <- predict(nsq_glm_veg_noRE,
                              newdata = nsq_data_floral,
                              type = "zlink",
                              se.fit = TRUE)

# Combine predictions and transform to response scale (by applying the inverse logit (plogit))
pred_nsq_floral_df <- cbind(
  nsq_data_floral,
  zi_fit = plogis(pred_nsq_floral_zi$fit),
  zi_lower = plogis(pred_nsq_floral_zi$fit - 1.96 * pred_nsq_floral_zi$se.fit),
  zi_upper = plogis(pred_nsq_floral_zi$fit + 1.96 * pred_nsq_floral_zi$se.fit)
)

#plot
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = zi_fit)) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = zi_lower, ymax = zi_upper), alpha = 0.2, color=NA) +
  labs(
       x = "Floral abundance (scaled)",
       y = "Probability of\nExcess Zeros") +
  theme_classic(base_size = 20)
```



Season

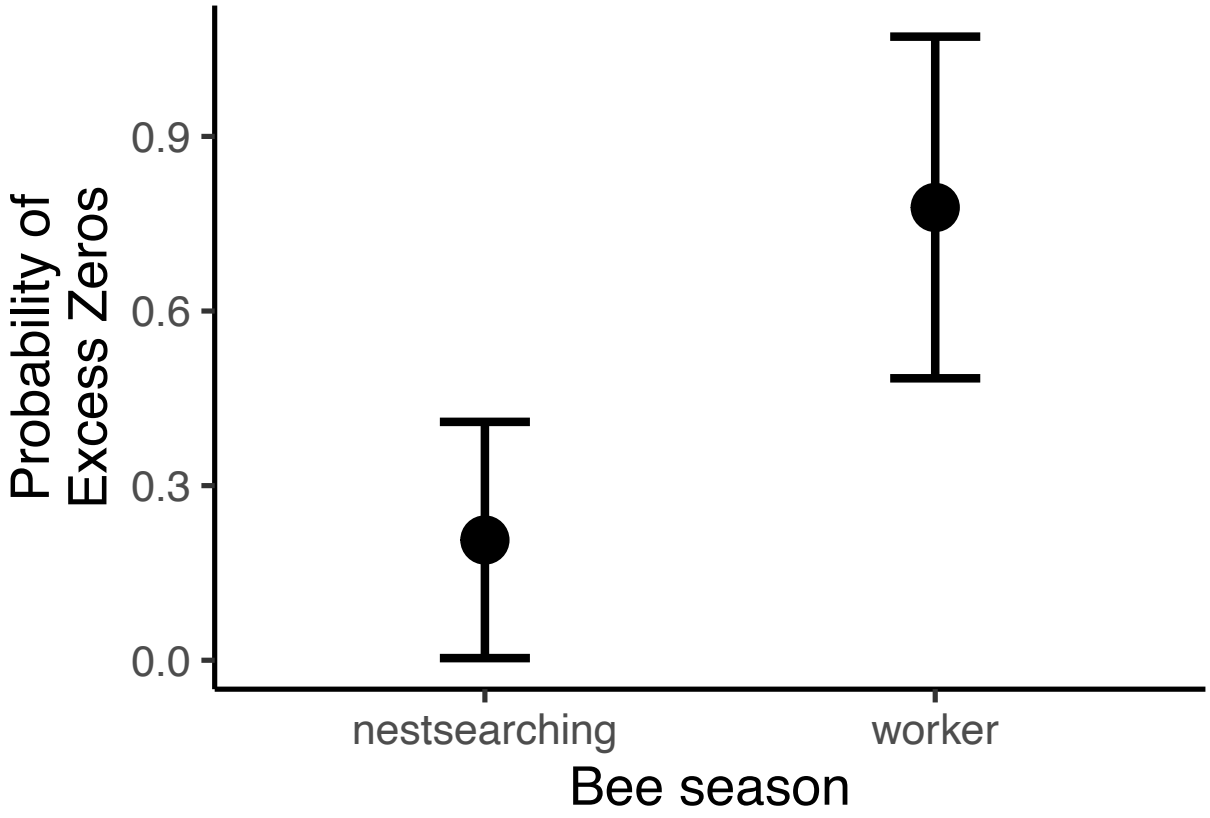
Conditional model:

```r
# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(surveys$bee_season_type)),
  plot_type = "ditch"
)

# Get predicted zero probabilities and standard errors
pred_nsq_season_cond <- predict(nsq_glm_veg_noRE,
                            newdata = nsq_data_season,
                            type = "link",
                            se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_season_df <- cbind(nsq_data_season,
                 cond_fit_log = pred_nsq_season_cond$fit,
                 cond_se_log = pred_nsq_season_cond$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_nsq_season_df <- pred_nsq_season_df %>%
  mutate(
    cond_fit = exp(cond_fit_log),
    cond_lower = exp(cond_fit_log - 1.96 * cond_se_log),
    cond_upper = exp(cond_fit_log + 1.96 * cond_se_log)
  )

ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = cond_lower, ymax = cond_upper), width = 0.2, position = position_dodge(0.6),
  labs(x = "Bee season",
       y = "Predicted mean NSQ\ncount (per plot)") +
  theme_classic(base_size = 20)
```



Zero-inflated model:

```
# Get predicted zero probabilities and standard errors
pred_nsq_season_zi <- predict(nsq_glm_veg_noRE,
                              newdata = nsq_data_season,
                              type = "zprob",
                              se.fit = TRUE)

# Combine predictions with data frame
pred_nsq_season_df <- cbind(nsq_data_season,
                  zi_fit = pred_nsq_season_zi$fit,
                  zi_se = pred_nsq_season_zi$se.fit)

#plot
ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = zi_fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), width = 0.2, position = pos
  labs(x = "Bee season",
       y = "Probability of\nExcess Zeros") +
  theme_classic(base_size = 20)
```



Findings

Conditional model (predicts abundance of nest-searching queens at sites where queens are present). Ditch is reference plot type.

- No significant effect of cover type, plot type, floral abundance, or season on NSQ abundance
- No significant interaction between cover type and plot type of floral abundance and plot type

Zero-inflation model (predicts presence/absence of nest-searching queens).

- NSQ are more likely to be absent from bare sites than grassy sites
- NSQ are more likely to be absent during worker season than NSQ season
- No significant effect of plot type or floral abundance on NSQ abundance
- No significant interaction between cover type and plot type of floral abundance and plot type

Workers

Steps

1) Try random effect structures with poisson.

- model with plot nested in site (no area) is best

```
workers_veg_full <- glmmTMB(workers ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                            bee_season_type+
                            (1|area/site_id/plot_id),
                          family = poisson,
                          data = surveys)

workers_veg_noArea <- glmmTMB(workers ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|site_id/plot_id),
                          family = poisson,
                          data = surveys)

workers_veg_siteOnly <- glmmTMB(workers ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|site_id),
                          family = poisson,
                          data = surveys)
workers_veg_noPlot <- glmmTMB(workers ~ cover_type*plot_type +
                              bombus_floral_abun_scaled*plot_type +
                              bee_season_type +
                            (1|area/site_id),
                          family = poisson,
                          data = surveys)

#compare models
anova(workers_veg_full, workers_veg_noArea, workers_veg_siteOnly, workers_veg_noPlot)
```

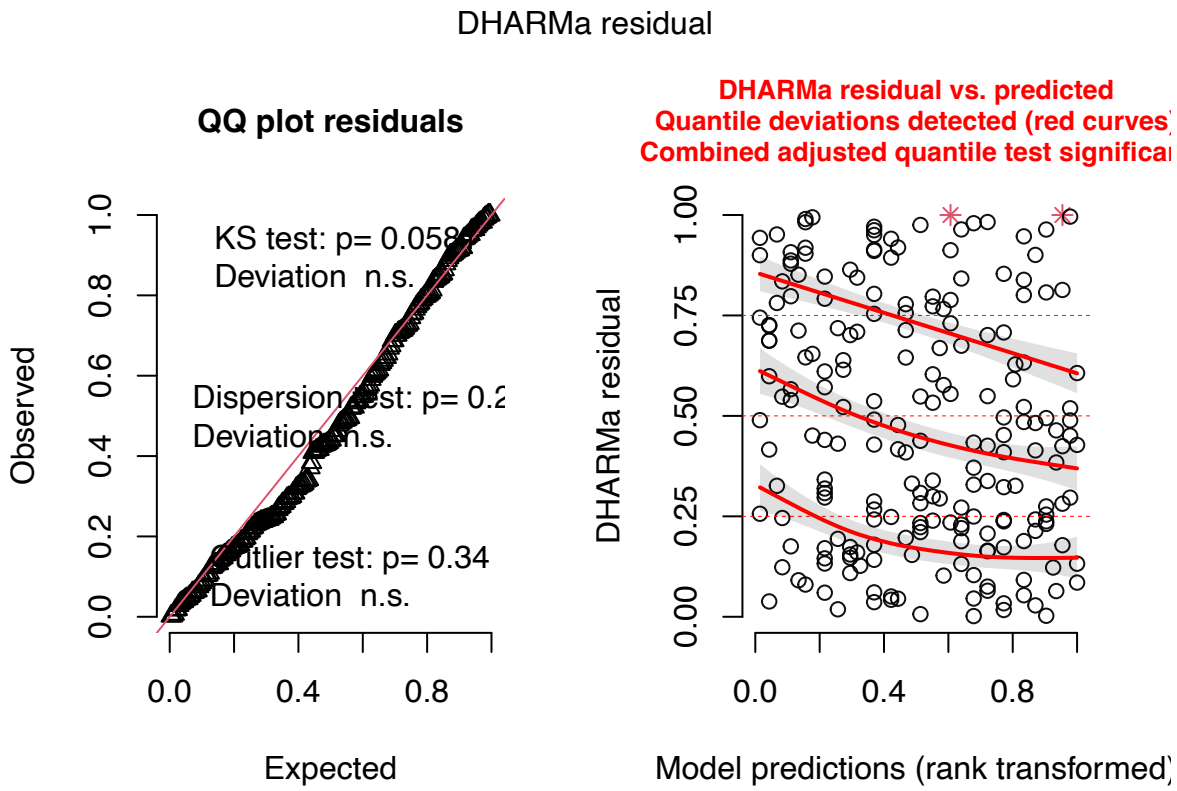
```
## Data: surveys
## Models:
## workers_veg_siteOnly: workers ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp=~
## workers_veg_siteOnly:     plot_type + bee_season_type + (1 | site_id), zi=~0, disp=~1
## workers_veg_noArea: workers ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp=~1
## workers_veg_noArea:     plot_type + bee_season_type + (1 | site_id/plot_id), zi=~0, disp=~1
## workers_veg_noPlot: workers ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp=~1
## workers_veg_noPlot:     plot_type + bee_season_type + (1 | area/site_id), zi=~0, disp=~1
## workers_veg_full: workers ~ cover_type * plot_type + bombus_floral_abun_scaled * , zi=~0, disp=~1
## workers_veg_full:     plot_type + bee_season_type + (1 | area/site_id/plot_id), zi=~0, disp=~1
##                       Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## workers_veg_siteOnly  8 330.88 357.65 -157.44   314.88
## workers_veg_noArea    9 330.49 360.61 -156.25   312.49 2.3849      1     0.1225
## workers_veg_noPlot    9 332.59 362.71 -157.29   314.59 0.0000      0     1.0000
## workers_veg_full     10 332.20 365.67 -156.10   312.20 2.3900      1     0.1221
```

```
##no area is best
```

2) Check diagnostics

- looks okay

```
residuals_workers_veg_noArea <- simulateResiduals(fittedModel = workers_veg_noArea, plot = TRUE)
```



3. Plot results.

```
summary(workers_veg_noArea)
```

```
##  Family: poisson  ( log )
## Formula:
## workers ~ cover_type * plot_type + bombus_floral_abun_scaled *
##     plot_type + bee_season_type + (1 | site_id/plot_id)
## Data: surveys
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     330.5     360.6    -156.2     312.5       201
##
## Random effects:
##
## Conditional model:
##  Groups          Name        Variance Std.Dev.
##  plot_id:site_id (Intercept) 0.3032   0.5506
##  site_id         (Intercept) 0.7192   0.8481
## Number of obs: 210, groups:  plot_id:site_id, 36; site_id, 12
##
## Conditional model:
```

```
##                                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                             -2.9119388  0.6582799  -4.424 9.71e-06
## cover_typegrass                         -0.1827406  0.7166995  -0.255   0.7987
## plot_typefield                          -1.0597605  0.4704448  -2.253   0.0243
## bombus_floral_abun_scaled                0.0004254  0.2188336   0.002   0.9984
## bee_season_typeworker                    2.5685905  0.4332806   5.928 3.06e-09
## cover_typegrass:plot_typefield           0.3112268  0.6688794   0.465   0.6417
## plot_typefield:bombus_floral_abun_scaled 0.7418576  0.3106416   2.388   0.0169
##
## (Intercept)                             ***
## cover_typegrass
## plot_typefield                          *
## bombus_floral_abun_scaled
## bee_season_typeworker                   ***
## cover_typegrass:plot_typefield
## plot_typefield:bombus_floral_abun_scaled *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effect of floral abundance within each plot type

```
#calculate the slope of floral abundance for each plot type
eff_slopes <- emtrends(workers_veg_noArea, ~ plot_type, var = "bombus_floral_abun_scaled")

#test if those slopes are significantly different from zero
summary(eff_slopes, infer = c(TRUE, TRUE))
```

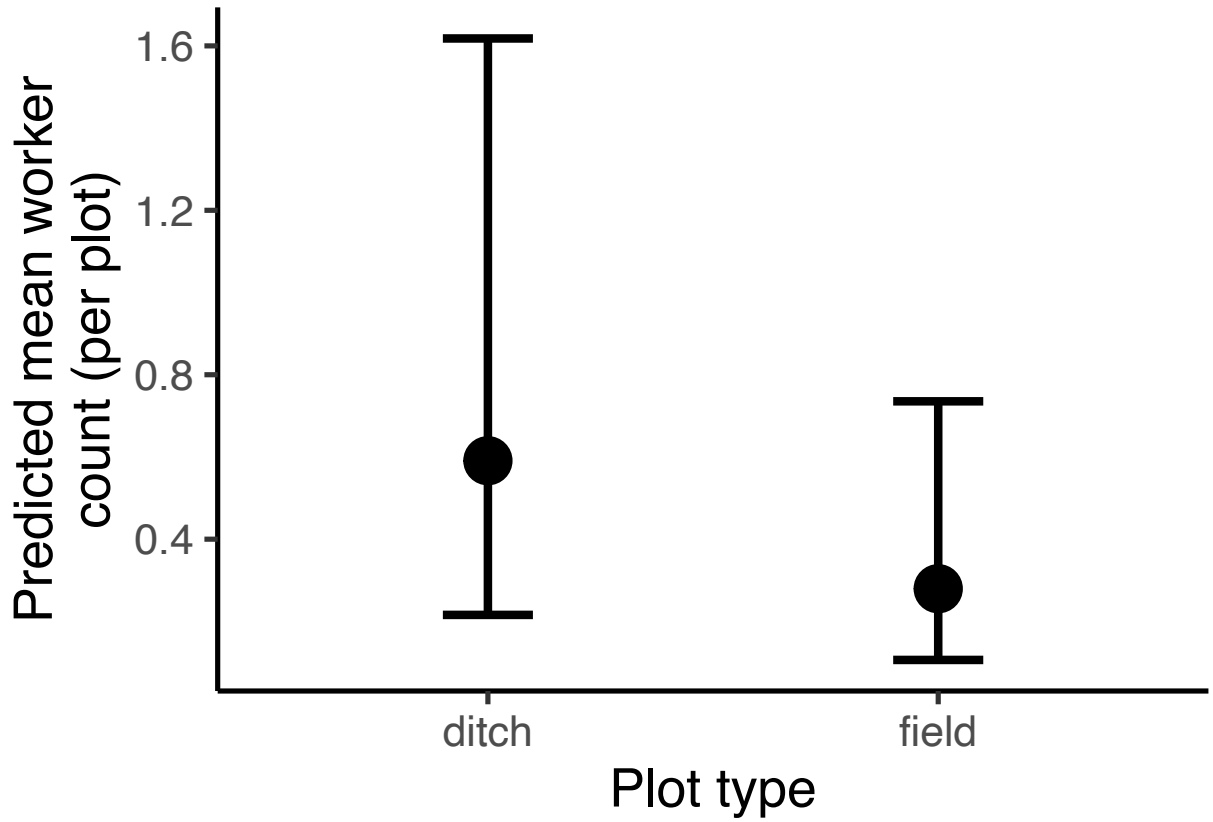
```
##  plot_type bombus_floral_abun_scaled.trend    SE  df asymp.LCL asymp.UCL
##  ditch                             0.000425 0.219 Inf    -0.428     0.429
##  field                             0.742283 0.218 Inf     0.315     1.170
##  z.ratio p.value
##    0.002  0.9984
##    3.401  0.0007
##
## Results are averaged over the levels of: cover_type, bee_season_type
## Confidence level used: 0.95
```

Plots

Plot type

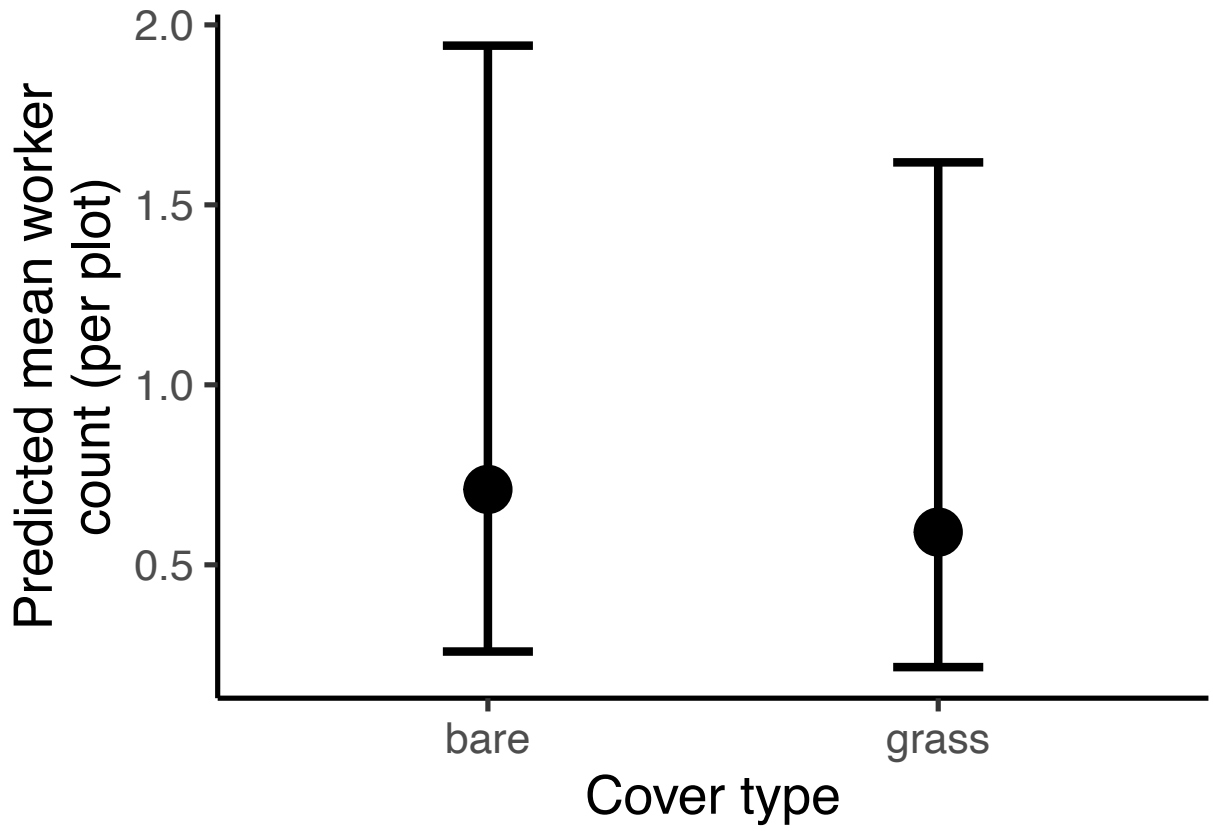
```
# Create newdata
worker_data_plottype <- expand.grid(
  cover_type = "grass",
  plot_type = factor(c("ditch", "field"), levels = levels(surveys$plot_type))
)
worker_data_plottype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
worker_data_plottype$bee_season_type <- "worker"

# Get predicted probabilities and standard errors
pred_worker_plottype <- predict(workers_veg_noArea,
                           newdata = worker_data_plottype,
```

```
                            se.fit = TRUE,
                            re.form = NA,
                            type = "link")

# Combine predictions with data frame
pred_worker_plottype_df <- cbind(worker_data_plottype,
                 fit_log = pred_worker_plottype$fit,
                 se_log = pred_worker_plottype$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_plottype_df <- pred_worker_plottype_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_plottype_df, aes(x = plot_type, y = fit)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Plot type",
       y = "Predicted mean worker\ncount (per plot)")  +
  theme_classic(base_size = 20)
```



Cover type

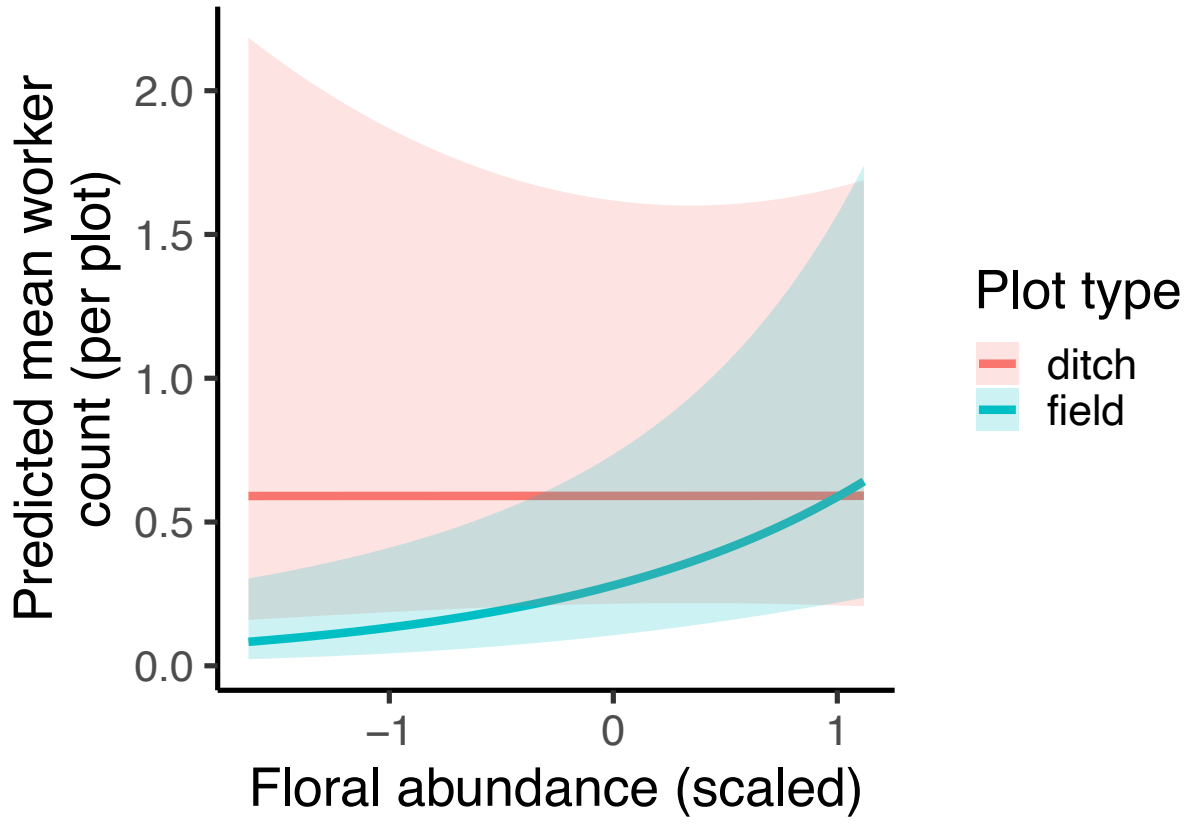
```r
# Create newdata
worker_data_covertype <- expand.grid(
  cover_type = factor(c("grass", "bare"), levels = levels(surveys$cover_type)),
  plot_type = "ditch"
)
worker_data_covertype$bombus_floral_abun_scaled <- mean(surveys$bombus_floral_abun_scaled)
worker_data_covertype$bee_season_type <- "worker"

# Get predicted probabilities and standard errors
pred_worker_covertype <- predict(workers_veg_noArea,
                            newdata = worker_data_covertype,
                            se.fit = TRUE,
                            re.form = NA,
                            type = "link")

# Combine predictions with data frame
pred_worker_covertype_df <- cbind(worker_data_covertype,
                 fit_log = pred_worker_covertype$fit,
                 se_log = pred_worker_covertype$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_covertype_df <- pred_worker_covertype_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_covertype_df, aes(x = cover_type, y = fit)) +
  geom_point(size = 8, position = position_dodge(0.6)) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Cover type",
       y = "Predicted mean worker\ncount (per plot)") +
  theme_classic(base_size = 20)
```



Floral abundance x plot type

```
#create new data grid
worker_data_floral<- expand.grid(
  cover_type = "grass",
  plot_type = unique(surveys$plot_type),
  bombus_floral_abun_scaled = floral_seq,
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_floral <- predict(workers_veg_noArea,
                              newdata = worker_data_floral,
                              se.fit = TRUE,
                              re.form = NA,
                              type = "link")

# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                 fit_log = pred_worker_floral$fit,
                 se_log = pred_worker_floral$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_floral_df <- pred_worker_floral_df %>%
  mutate(
    fit = exp(fit_log),
```

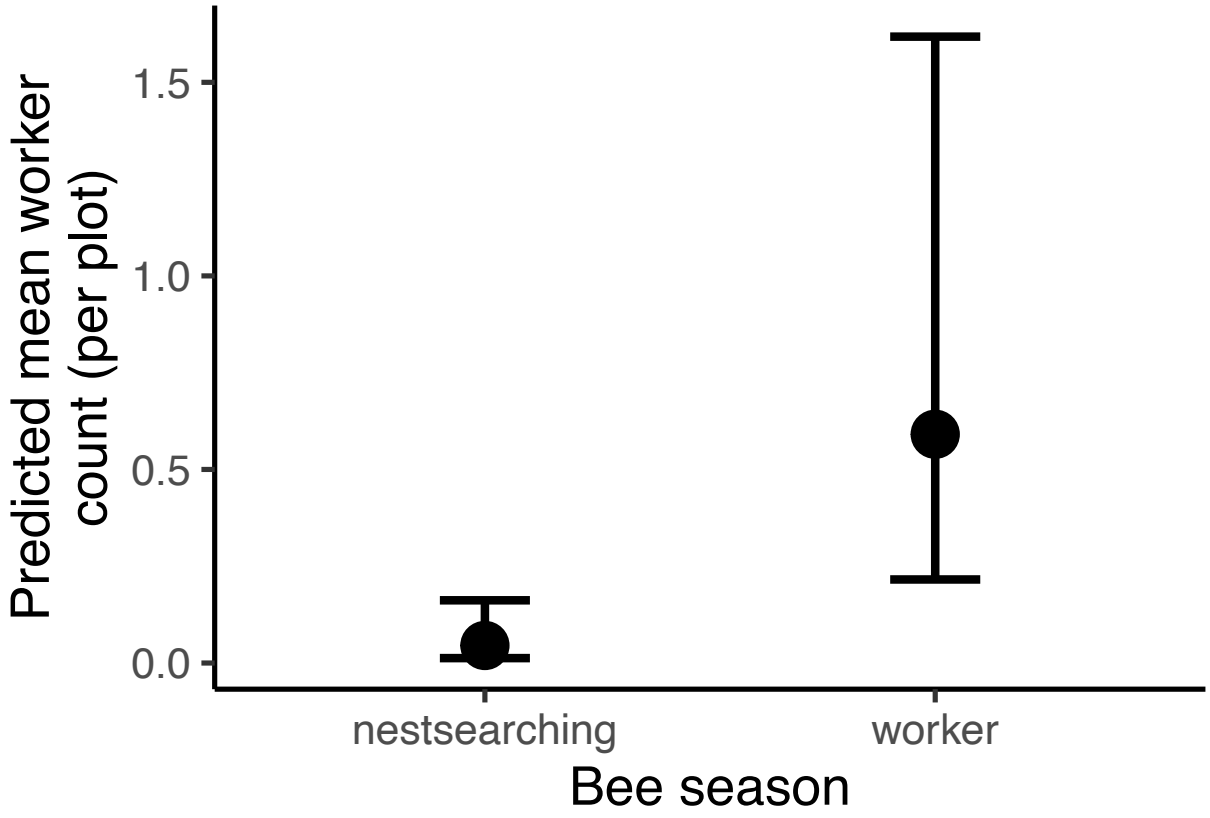
```
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit, color = plot_type)) +
  geom_line(linewidth = 1.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper, fill = plot_type), alpha = 0.2, color=NA) +
  labs(x = "Floral abundance (scaled)",
       y = "Predicted mean worker\ncount (per plot)",
       fill = "Plot type",
       color = "Plot type") +
  theme_classic(base_size = 20)
```



Season

```
#create new data grid
worker_data_season<- expand.grid(
  cover_type = "grass",
  plot_type = "ditch",
  bombus_floral_abun_scaled = mean(surveys$bombus_floral_abun_scaled, na.rm = TRUE),
  bee_season_type = unique(surveys$bee_season_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(workers_veg_noArea,
```

```
                            newdata = worker_data_season,
                            se.fit = TRUE,
                            re.form = NA,
                            type = "link")

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
                 fit_log = pred_worker_season$fit,
                 se_log = pred_worker_season$se.fit)

#calculate predictions and confidence intervals on response scale (not log scale)
pred_worker_season_df <- pred_worker_season_df %>%
  mutate(
    fit = exp(fit_log),
    lower = exp(fit_log - 1.96 * se_log),
    upper = exp(fit_log + 1.96 * se_log)
  )

#Plot
ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_point(size = 8) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2, position = position_dodge(0.6), linewidth
  labs(x = "Bee season",
       y = "Predicted mean worker\ncount (per plot)",
       fill = "Plot type",
       color = "Plot type") +
  theme_classic(base_size = 20)
```



Findings

Cover type

- No effect of cover type on worker abundance (p=0.80)
- More workers along the ditch than within the field (p=0.0243)
- No interaction between cover type and plot type

Floral abundance

- No overall impact of floral abundance on workers (p=0.9984)
- Significant interaction between plot type and floral abundance (p=0.0169)
- Floral abundance influences worker abundance within the field but NOT along the ditch

Date

- Significantly more workers in worker season that NS (p<0.0001)